LINFO2145 — Cloud Computing

Lesson 2: Platform-as-a-Service and service-oriented architectures



Pr. Etienne Rivière
etienne.riviere@uclouvain.be

Announcements

- There is no lecture next week
  - Next lecture is on October 8th, 10:45
- The project will start this Friday!
  - ULB students without Moodle access: give your GitHub account to TAs by email or in-person
  - You can announce your group when you get Moodle access (but must not work with others than your chosen teammate)
  - Today's lecture is arguably the most critical for the project
  - We may go over left-over material from lecture 1 at the end but only if time allows (nothing critical)
- Quiz #1 is still open for one week (deadline Oct. 1st, 10:45)
  - Peer review will then start for students having submitted an answer (deadline Oct. 8th, 10:45)

Course objectives

- Discuss representative Platform-as-a-Service
  - Development environments
  - "Serverless" Cloud computing

- Describe service-oriented architectures
  - "Classical" Web services
  - REST APIs
  - Microservices
  - API gateway and Service Mesh

1st part: Platform-as-a-Service

Platform-as-a-Service

- A cloud service model providing developers with a managed environment where they can create and deploy applications
  - ☑ Relief from administration and maintenance aspects
    - No need to install and maintain OS, database server, web server, etc. by hand (this is just overhead for developers)
  - ☐ Less flexibility than IaaS
    - Must accommodate options offered by PaaS provider
  - ☐ Greater *vendor lock-in* than with IaaS
    - Easier to relocate VMs or containers between IaaS providers than code written for a specific PaaS platform

Some of the many PaaS

Google App Engine



heroku

appfog

Engine Yard™

redhat. OPENSHIFT

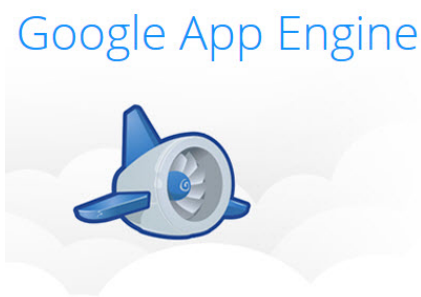
clevercloud

Comparing PaaS

- PaaS more complicated to compare than IaaS
  - No longer *only* about cost, machine performance, or security features
  - Each development and application creation environment has its pros and cons
- Some criteria
  - Supported programming language and libraries
  - Availability of database management systems
  - Communication facilities (between apps, with users, etc.)
  - Deployment and testing support
  - Cost and billing model (may be less predictable than IaaS)

Example: Google App Engine

- Completely managed environment
  - Application runtime, libraries, OS, development tools
  - Target: distributed Web applications
  - Java, Ruby, Python, PHP, Go, and Node.js



- Target = applications with low response time, split in small and stateless components
  - Components called only when incoming web request
  - State kept in a managed database
- Also possible to mix IaaS-like usage by providing containers with arbitrary code/functions
  - Management using Kubernetes (container management platform)

Google App Engine: pricing

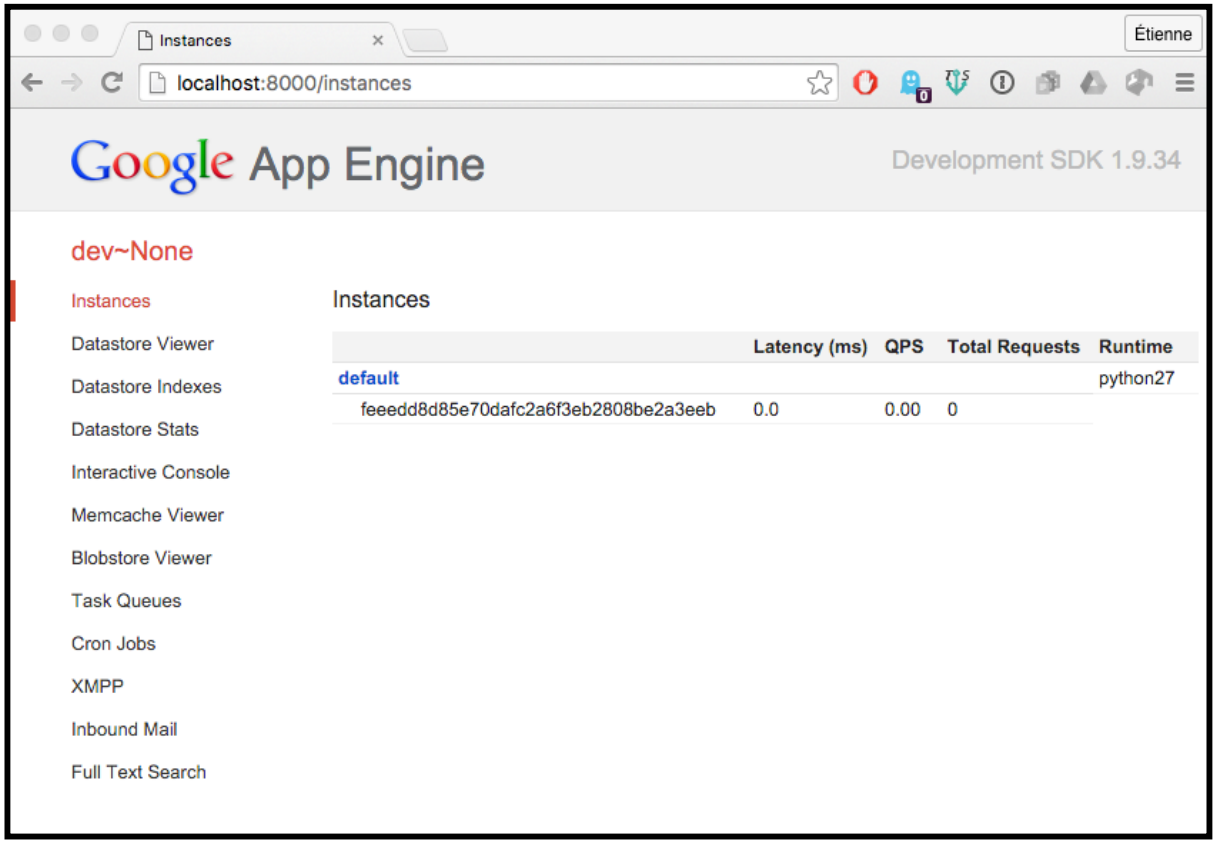
- Billing based on actual usage, billed per second (with a minimum of one minute billed per hour)

Resource	Unit	Unit cost
vCPU	per core hour	$0.0526
Memory	per GB hour	$0.0071
Persistent disk	per GB per month	$0.0400

- Cost for outgoing traffic: $0.12 / GB
- Shared Google services (caching, email, libraries): no extra charge—good for vendor lock-in

Example: deploying a hello world server on GAE

- Example in Python (from Google)
- Provided SDK (*Software Development Kit*)
  - Support local execution of applications for testing



Python (deprecated version) code

```
import webapp2
```

← the Google-provided web application framework

```
class MainPage(webapp2.RequestHandler):
    def get(self):
        self.response.headers['Content-Type'] = 'text/plain'
        self.response.write('Hello, World!')

app = webapp2.WSGIApplication([
    ('/', MainPage),
], debug=True)
```

Configuration file (YAML format)

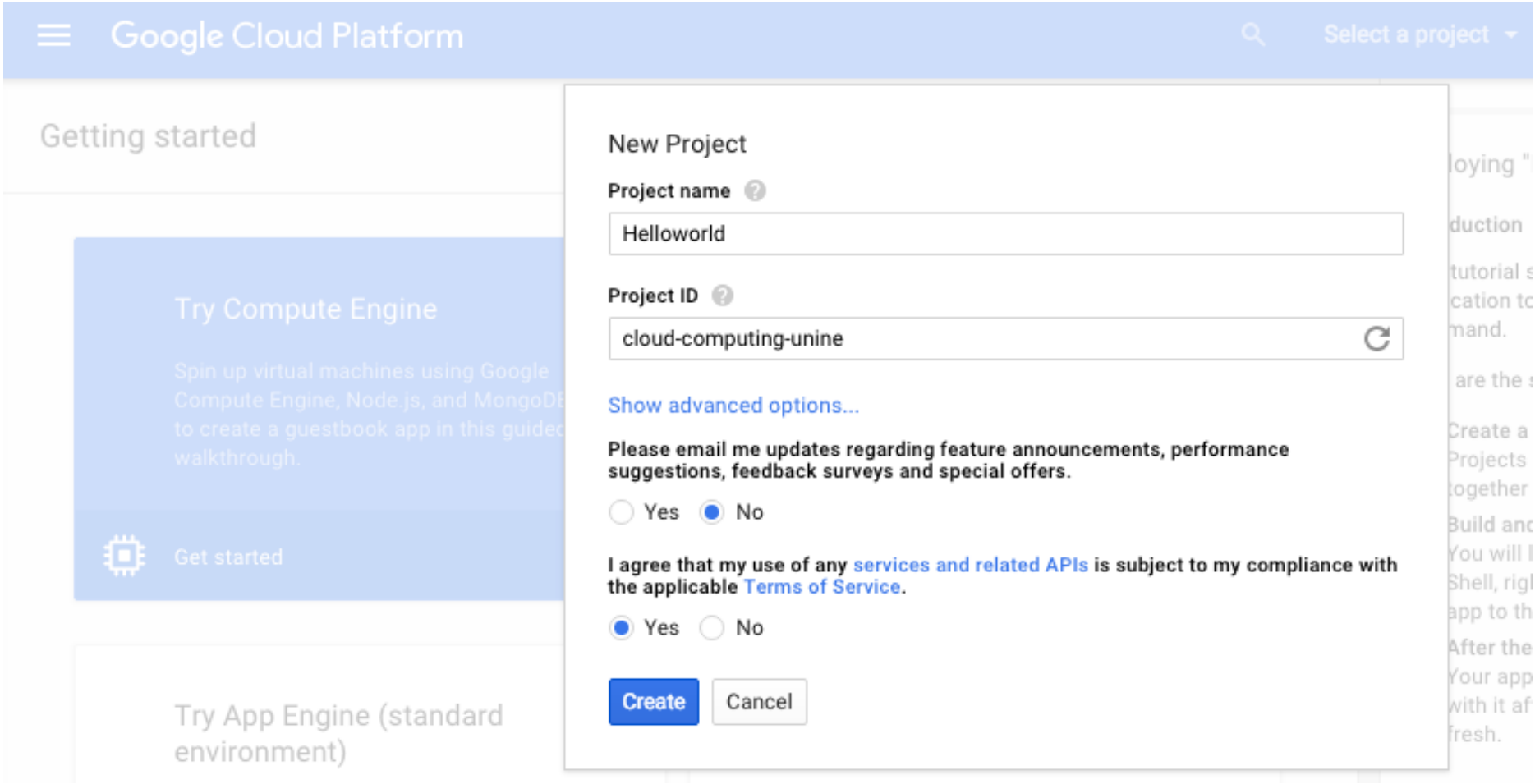
```
runtime: python27
api_version: 1
threadsafe: true

handlers:
- url: /.*
  script: helloworld.app
```

same object can serve multiple // requests?

regexp: all URLs handled by app object in helloworld module

Deployment: create project



Deployment: upload project

tmp — -bash — 82×34

```
<Etiennes-iMac:etienne> 0 [03-12 14:55] tmp (37.436 Mb)
! appcfg.py -A cloud-computing-unine -V v1 update helloworld/
02:55 PM Application: cloud-computing-unine (was: None); version: v1 (was: None)
02:55 PM Host: appengine.google.com
02:55 PM
Starting update of app: cloud-computing-unine, version: v1
02:55 PM Getting current resource limits.
Your browser has been opened to visit:

    https://accounts.google.com/o/oauth2/auth?scope=https%3A%2F%2Fwww.googleapis.c
om%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-platfo
rm+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email&redirect_uri=http%3A%2
F%2Flocalhost%3A8080%2F&response_type=code&client_id=550516889912.apps.googleuserc
ontent.com&access_type=offline

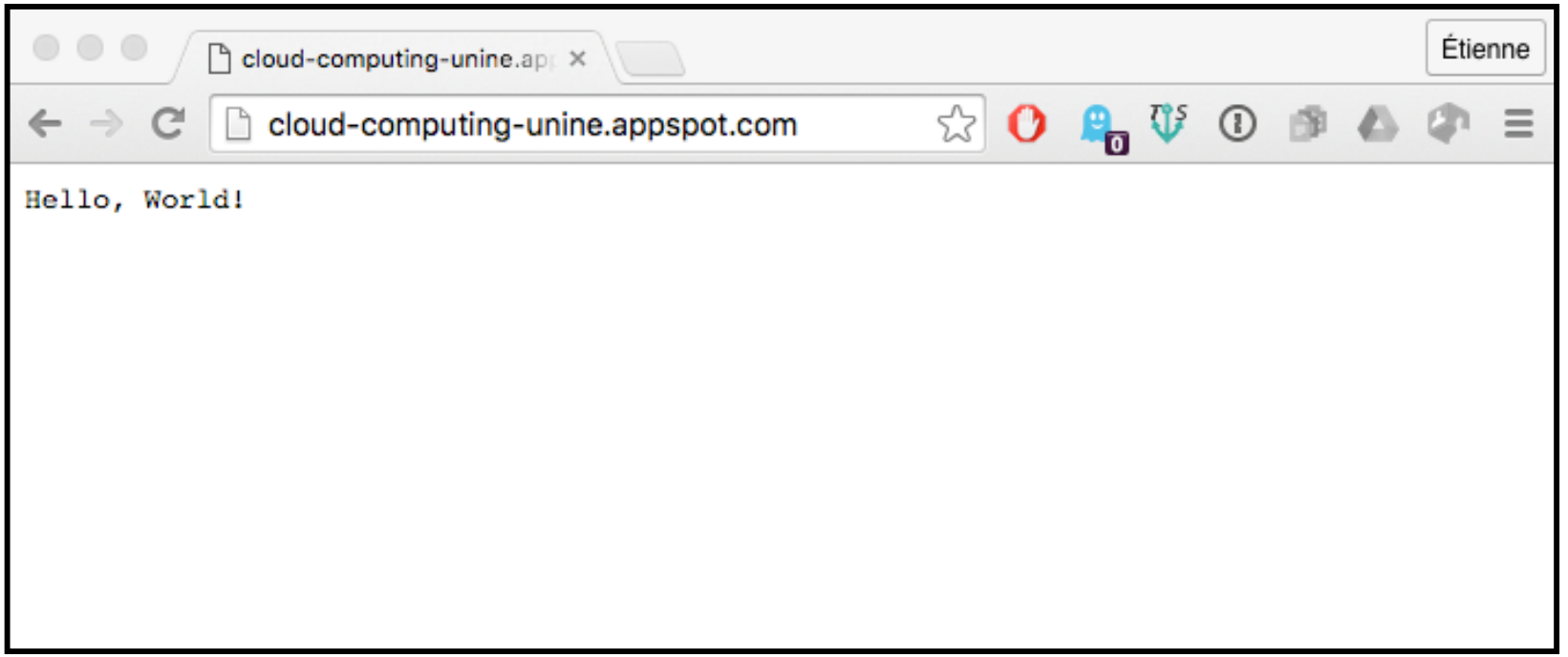
If your browser is on a different machine then exit and re-run this
application with the command-line parameter

  --noauth_local_webserver

Authentication successful.
02:55 PM Scanning files on local disk.
02:55 PM Cloning 2 application files.
02:55 PM Compilation starting.
02:55 PM Compilation completed.
02:55 PM Starting deployment.
02:55 PM Checking if deployment succeeded.
02:55 PM Deployment successful.
02:55 PM Checking if updated app version is serving.
02:55 PM Completed update of app: cloud-computing-unine, version: v1

<Etiennes-iMac:etienne> 0 [03-12 14:55] tmp (37.436 Mb)
!
```

Et voila


cloud-computing-unine.ap
Étienne
cloud-computing-unine.appspot.com
Hello, World!

Google AppEngine and portability

- What if
  - I want to run my application built using GAE in my private cloud instead of Google's data centers?
  - I want to move to an IaaS platform for costs reasons
- 

  - Open source alternative to run Google App Engine code on a self-maintained infrastructure
  - Runs on the WildFly application server (part of Red Hat JBoss Middleware)

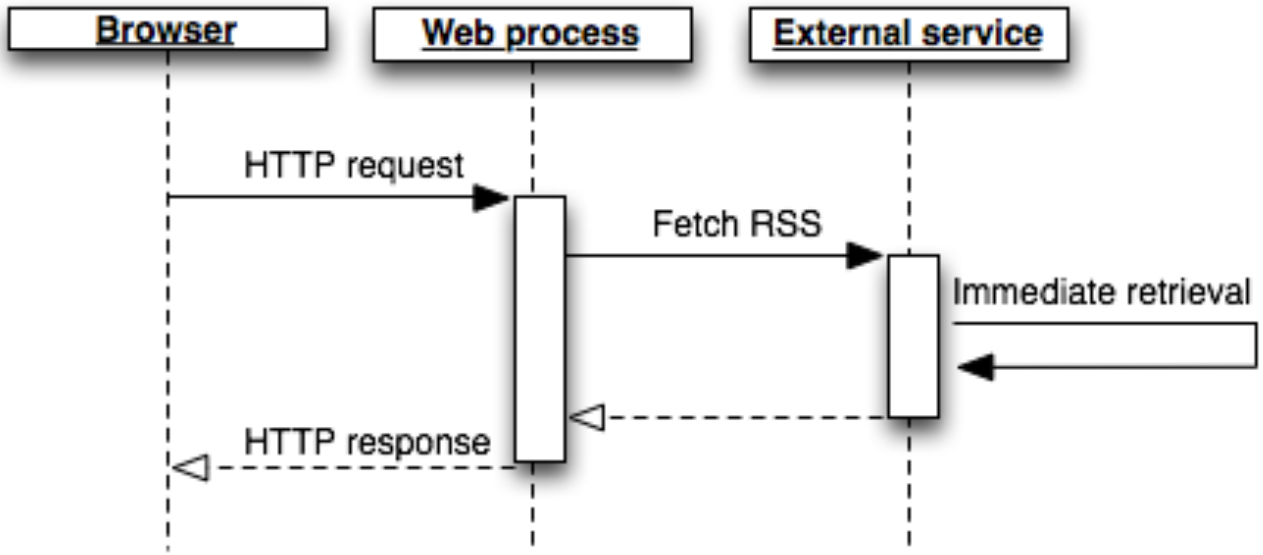
"Serverless" Computing

- Google App Engine does not require the user to manage a specific VM or container
  - But the code does not run out of thin air …
  - … the actual server on which they run is just not known (or of any importance) to the user
- The most common form of PaaS, named "*serverless*" computing
- Other examples
  - Heroku
  - Amazon AWS *lambdas*
  - Azure functions



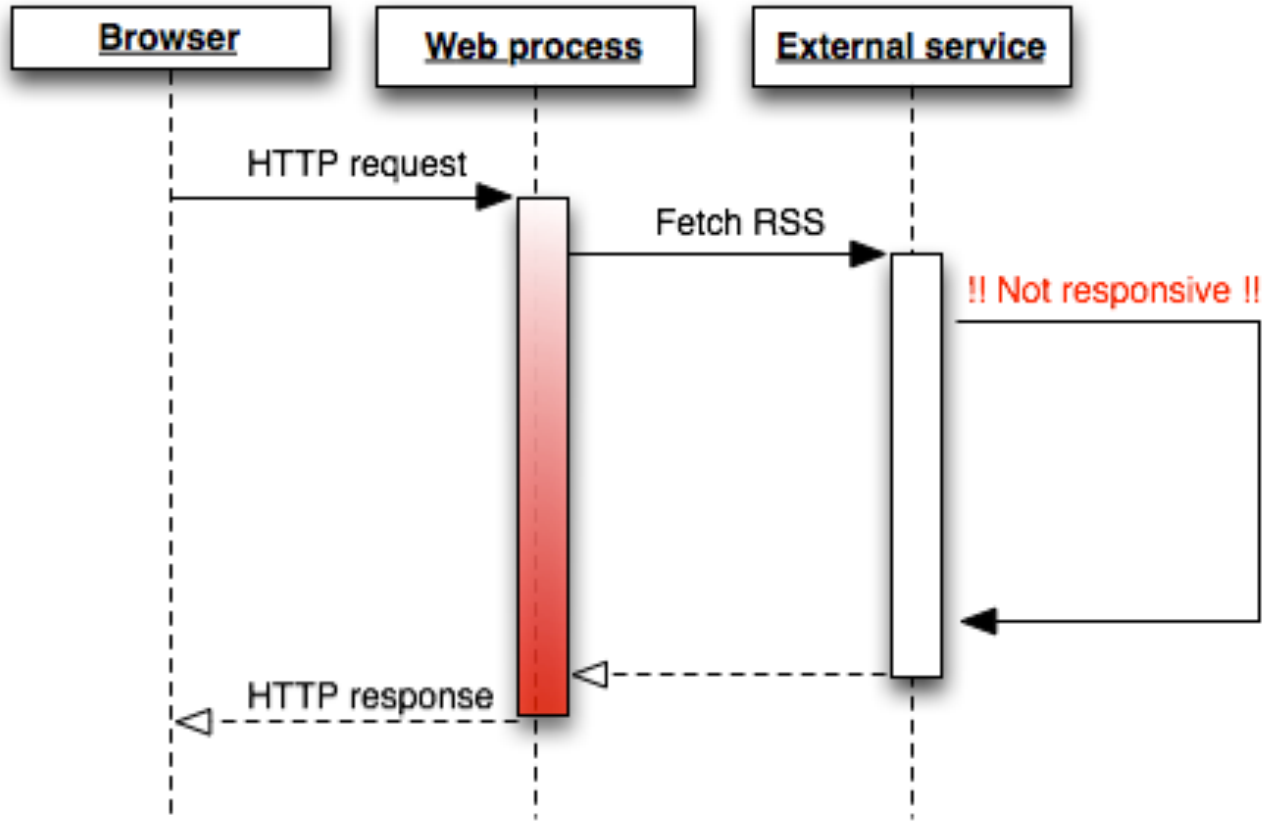
- One of the first PaaS
  - Started as a platform for Ruby developers
  - Now supports Python, Java, Scala, Clojure, node.js, etc.
- *dynos:* serverless computing components
  - Web dynos: respond to HTTP requests
  - Worker dynos: execute requests from *task queue*
    - Transfer computationally intensive tasks outside the Web layer and the user request/response cycle
    - A typical design strategy
  - Actually running in containers
    - Choice between on-demand (cheap, high latency) and persistent (more expensive, better response times)

Web/Worker dynos in Heroku



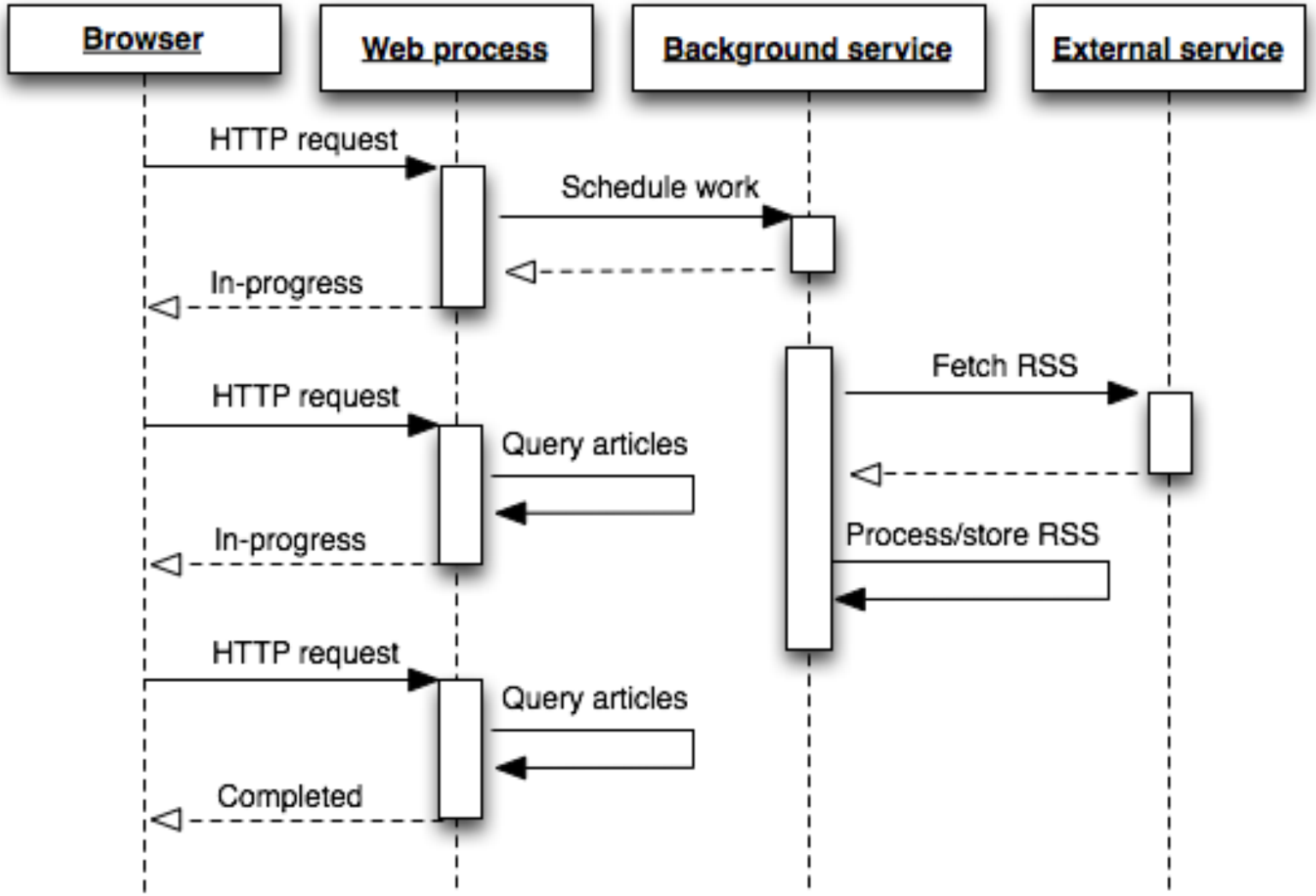
source: Heroku online documentation

Web/Worker dynos in Heroku



source: Heroku online documentation

Web/Worker dynos in Heroku



source: Heroku online documentation



- Support for deployment: integration with GitHub
  - Ability to 'push' an application to Heroku directly using github
  - Continuous Integration & Continuous Delivery/Deployment (CI/CD)
- Relatively limited set of managed support services
  - Storage: PostgreSQL or REDIS (*a key/value store*)
  - Communication: Apache Kafka messaging service
- Dyno pricing is per second (up to $25-$500 per month)
- Plus price of managed services
  - e.g. PostgreSQL: $50 to $4,500 /month depending on DB size

AWS lambdas

- Mechanism similar to Heroku's dynos
  - Register a piece of code that runs when an event triggers
    - Periodically
    - or for an incoming HTTP request
    - or upon communication from other components of your Cloud application
- Typical use: pair with communication service
  - Amazon Simple Queue Service (SQS) or Amazon Simple Notification Service (SNS)
  - Pushing 'work' in a named communication queue triggers lambdas that can consume/execute the order
    - One-to-many *decoupled* communication: favors extensibility

AWS lambdas (continued)

- Limitations per running instance
  - ~3GB memory, 512 MB temporary disk
  - 300 seconds execution
- 1.000 max concurrent instances executions
- Languages: node.js, Java, Python, Go, C#
  - Example in Python:




```
def my_handler(event, context):
    message = 'Hello {} {}!'.format(event['first_name'],
                                    event['last_name'])
    return {
        'message' : message
    }
```

Image thumbnails: the 'hello world' of lambdas

- Amazon S3 is a key/value storage service
  - Simple interface: add an element to a *bucket* under a key (unique name); retrieve element by its key
- Imagine a social media application
  - Users upload photos
  - We wish to create thumbnails or resize large photos for optimal web display
  - This can take time
    - Should not be part of client event-response loop!
  - Instead, store raw photo in a S3 bucket and return

High-level architecture

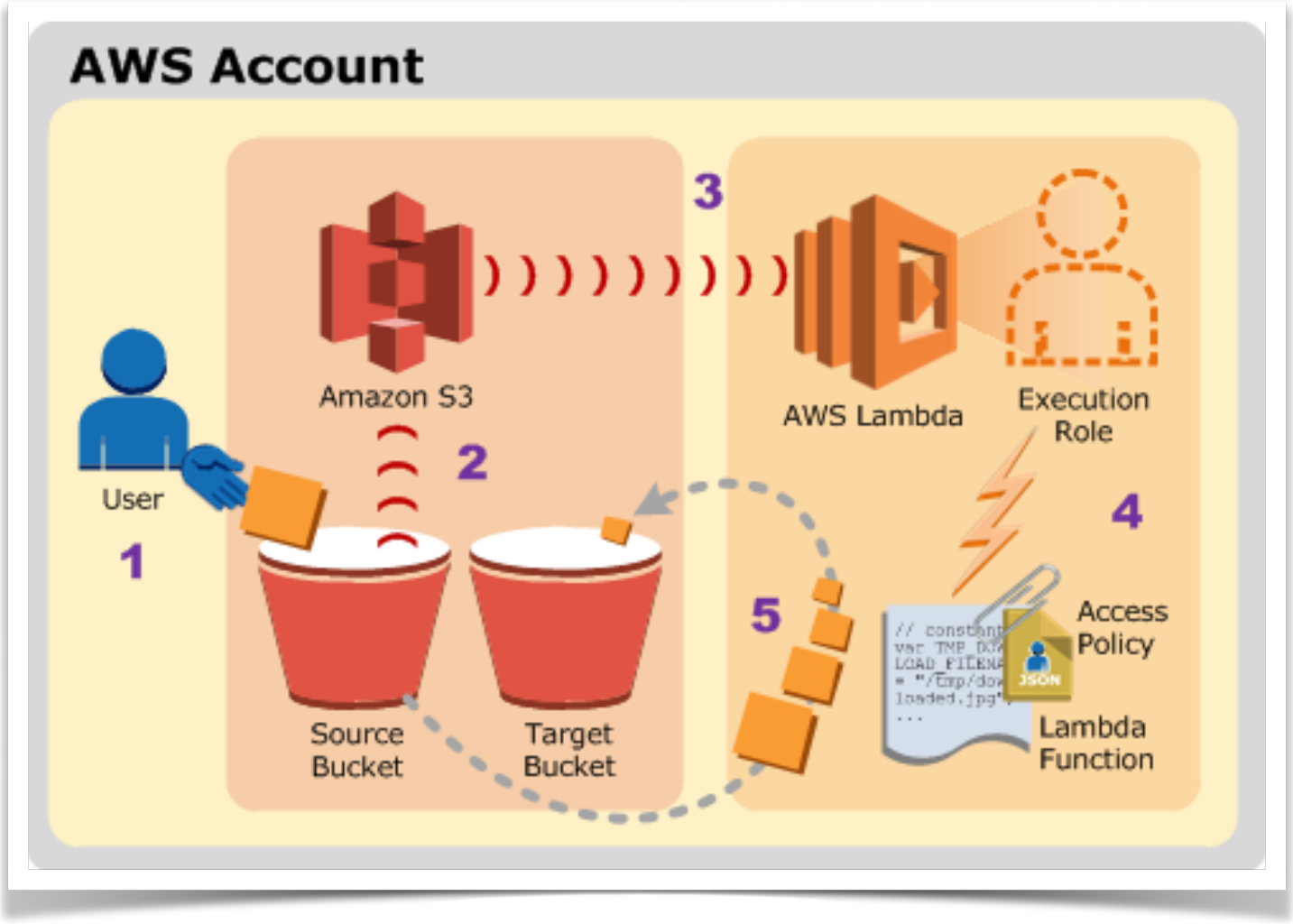


image from Amazon documentation

Steps

- A user uploads an image to the source bucket
- This generates an *event* of type `s3:ObjectCreated:*`
- Automatically calls the lambda that was previously registered for this type of event, on this specific bucket
- The lambda resizes the image (e.g. using an existing library) and saves it to another bucket using the S3 API
  - Why don't we use the same bucket?
- Azure Functions follow the same principles as AWS lambdas
  - Tutorials available online
  - Optional project feature for resizing new products' images can (but do not have to) use Azure Functions, can give bonus points

Intermediate conclusion

- Design principles in PaaS and serverless approaches around the notion of services
  - Called upon an event (lambda)
  - Upon some HTTP request from the client
- This is typical of a *service-oriented* architecture
  - Application is formed of a collection of services
  - Services interact using their APIs, or using middleware support implementing decoupled communication via events
  - Well adapted to Cloud deployments, where services can be scaled and deployed independently on many machines
- In the following, we will review service-oriented architectures and design principles

2nd part: Service-oriented architectures, REST interfaces and microservices

Service-oriented architectures

- Applications formed of remote *Services* accessed by
  - Desktop of mobile native clients (*e.g.*, Android/iOS application)
  - Web client running in a browser
  - Other applications performing *service composition*

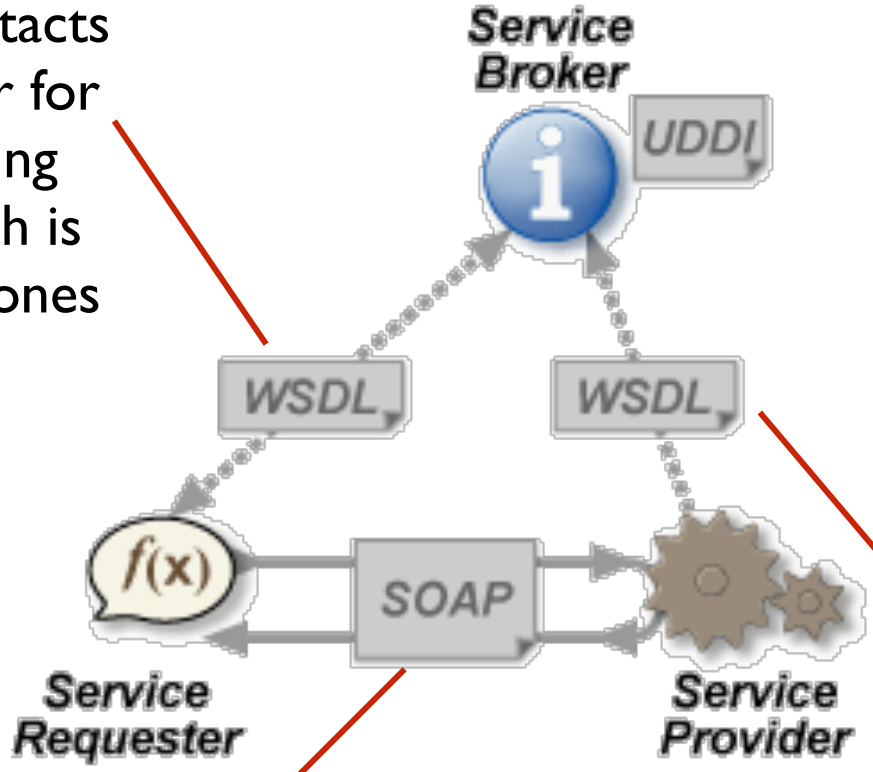
- What do we need?
  - Ability to discover available services
  - Data representation: formatting parameters and responses
    - Easy parsing, generation and format checking
    - Must be OS, platform and architecture independent
    - Bonus: human readability to ease development
  - Interaction with remote services using a standardized method

W3C Web Services

- Introduced in 2002 by the W3C
  - World Wide Web consortium, a standardization body
  - Sometimes called "Big" Web Services
- Components based on a set of open standards
  1. Interface to a Web Service (WS) described in a domain-specific description language, WSDL
  2. A special service, the UDDI, lists available services
     - Like DNS for HTTP addresses
  3. Connection to the WS with HTTP used as transport layer
  4. Interactions and data transfer with the WS using the SOAP interaction protocol
  5. Documents (request and response) formatted using XML according to a XSD format description

W3C Web Service interactions

2. Service requester contacts UDDI to locate provider for service it needs by sending appropriate WSDL, which is matched against stored ones



3. Service requester contacts service provider using SOAP protocol, exchanges use XML format representation and are checked against XSD files for wellformedness

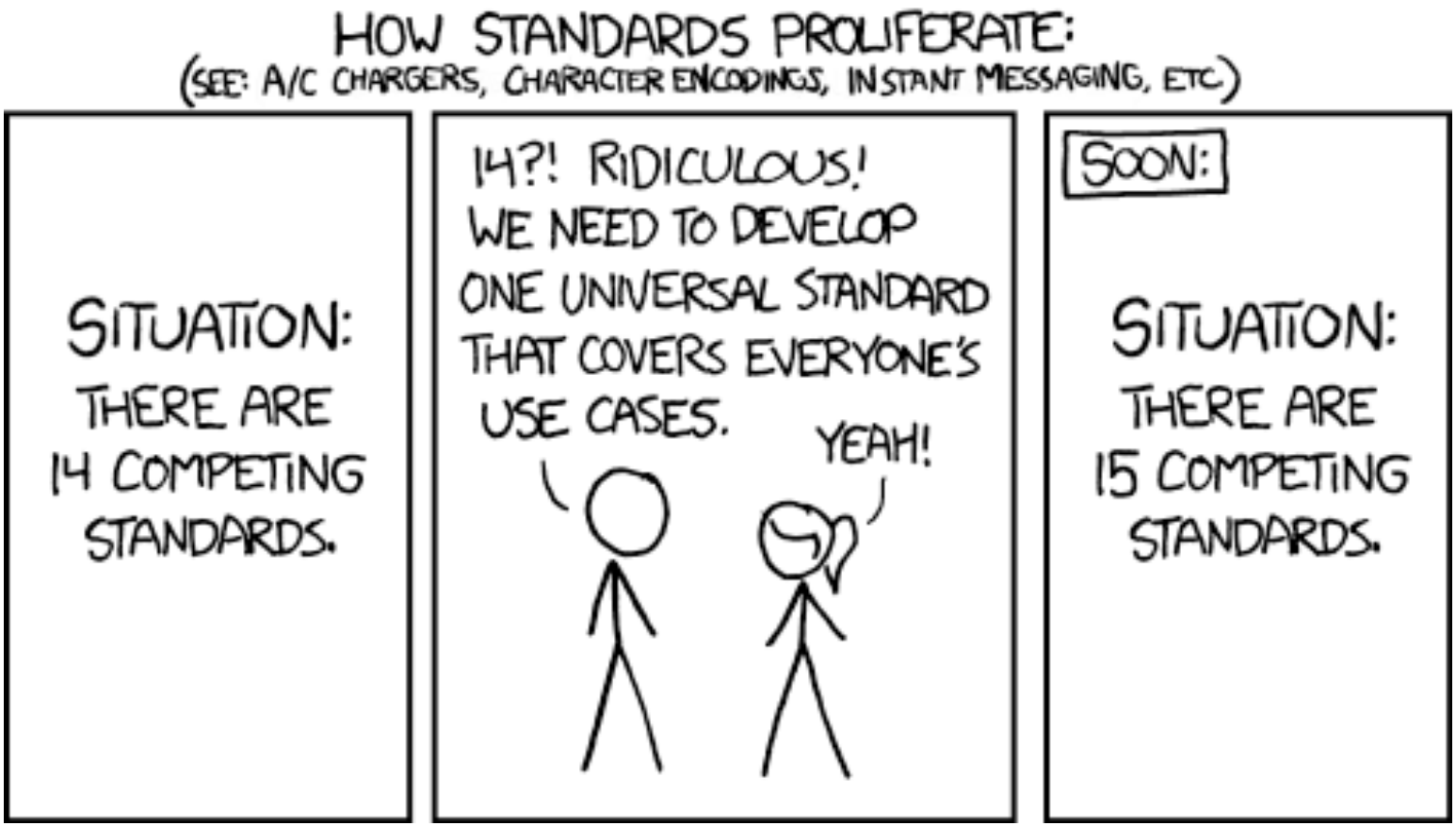
1. service provider announces its service by sending WSDL file to UDDI service broker

image by H.Voormann, CC BY-SA 3.0

Criticism of W3C/SOAP WS

- Complex and intricate to fully implement
- Use of XML and SOAP lead to poor performance
  - HTTP is only used as a transport layer, services descriptions and invocations in XML very verbose
- Service brokerage not really useful in practice
  - Developers know which service provider they want to connect to, rather than using any (unknown) service provider that happens to match a specification
- Failure to impose open standards led to a plethora of proprietary, closed solutions

About standards ...



REST interfaces

REST

- REpresentational State Transfer
  - Introduced in the early 2000s
  - General guidelines for designing Web architecture
    - Best practices, not a formal standardized protocol
    - Bottom-up normalization (rather than top-down)
  - Initial outcomes
    - HTTP 1.1
    - Uniform Resource Identifier (URI)
- REST Web services
  - Compared to W3C WS, use HTTP more directly

REST key features

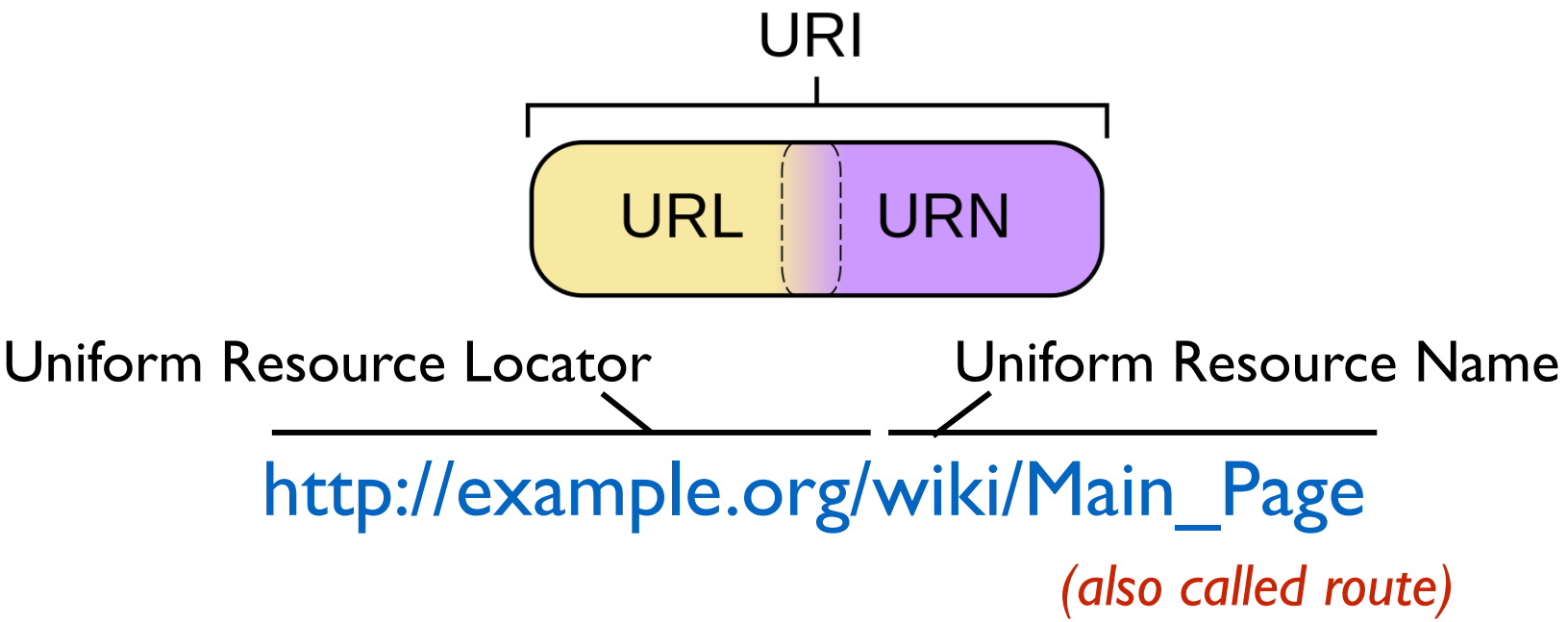
- Resource-based
- Representations
- Six constraints
  - Uniform interface
  - Statelessness
  - Client-Server
  - Cacheable
  - Layered System
  - Code on Demand

Resource-based

- Deal with 'things' rather than 'actions'
  - SOAP uses identified method calls (actions)
  - … whereas REST over HTTP calls a thing (resource) and applies a HTTP *verb* (GET/PUT/…) to that thing
    - Easier to determine impact on architecture of a call with a given verb
- Each thing identified by one (or more!) URI
  - **U**niversal **R**esource **I**dentifier
- A 'thing' is separate from its representations
  - Multiple representations possible for the same thing

URI

- Uniform Resource Identifier



Representations

- How resources are manipulated
  - When exchanged between client and server
  - How they are stored by either party does not matter
- Example
  - Resource: student (John)
  - Service: personal information (GET)
  - Representation
    - Name, program, list of pairs <course, grade>
    - JSON (or XML) format

Constraint 1: Uniform Interface

- All REST API follow a similar interface
- Based on HTTP verbs
  - GET, PUT, POST, PATCH, DELETE
  - Apply to resources identified by URIs
    - Options and parameters after ? symbol
    - `http://myserver.com/resourceX?option=true`
  - Responses in HTTP format
    - Header gives the *status* (success, errors, etc.)
    - Body

HTTP verbs: semantics

HTTP Verb	Effect
POST *Create*	Add a new element to a collection resource
GET *Read*	Returns a representation of resource • Safe method: should not modify anything on the server side • Does not produce any side effect (read-only)
PUT *Update/Replace*	• Replace existing version with provided representation • Delete existing version • Idempotent: produces the same result no matter how many times command is applied
DELETE	
PATCH Update/Modify	• Modify a resource existing on the server • Typically a diff between two representations: JSON Patch (or XML Patch) • Neither safe nor idempotent by default, need extra care to handle concurrent PATCH requests for same resource

HTTP verbs: example of use

HTTP Verb	Resource collection /students	Specific resource /students/john
POST *Create*	Create a new item in collection, e.g., /students/Mary Returns code 201 (Created)	not used
GET *Read*	Get list of students resource URIs (can also contain individual representations)	Get representation of single student
PUT *Update/Replace*	Replace entire collection by provided list of URIs	Replace a single student with provided representation
PATCH Update/Modify	not used	Update a specific student representation
DELETE	Delete complete list	Delete specific student resource

Design Effective & Safe APIs

blog.bytebytego.com



Use resource names (nouns)	GET /querycarts/123	GET /carts/123
Use plurals	GET /cart/123	GET /carts/123
Idempotency	POST /carts	POST /carts {requestId: 4321}
Use versioning	GET /carts/v1/123	GET /v1/carts/123
Query after soft deletion	GET /carts	GET /carts? includeDeleted=true
Pagination	GET /carts	GET /carts? pageSize=xx&pageToken=xx
Sorting	GET /items	GET /items? sort_by=time
Filtering	GET /items	GET /items? filter=color:red
Secure Access	X-API-KEY=xxx	X-API-KEY = xxx X-EXPIRY = xxx X-REQUEST-SIGNATURE = xxx hmac(URL + QueryString + Expiry + Body)
Resource cross reference	GET /carts/123? item=321	GET /carts/123/items/321
Add an item to a cart	POST /carts/123? addItem=321	POST /carts/123/items:add { itemId: "items/321" }
Rate limit	No rate limit - DDos	Design rate limiting rules based on IP, user, action group etc

HTTP responses

- Used to indicate status of request
- Status codes grouped in 5 families
  - 1xx : informational ;
  - 2xx : success ;
  - 3xx : redirection ;
  - 4xx : client error ;
  - 5xx : server errors.

Some common HTTP status codes

Code	Verb	Body content	Description
200 OK	GET, PUT, DELETE	Resource	Operation successful
201 Created	POST	Resource that was created	Successful creation
400 Bad request	all	Error message	Syntax error
401 Unauthorized	all	Error message	Action requires authentication
403 Forbidden	all	Error message	Invalid authentication
404 Not found	all	Error message	Resource not found
501 Not implemented	POST, PUT, DELETE	Error message	Req. HTTP op. not supported

JSON

```
{
    "firstName": "Robert",
    "lastName": "Meier",
    "age": 25,
    "registration": {
        "program": "JMCS",
        "year": "1"
    },
    "previousStudies": [
        {
            "program": "BSC",
            "institution": "UniFR",
            "grade": "5.5"
        },
        {
            "program": "Maturität",
            "institution": "Gymnasium Neufeld",
            "grade": "6"
        }
    ]
}
```

- Data format most commonly used for resource representations
  - *JavaScript Object Notation*
  - Human-readable, machine-parseable



Constraint 2: Statelessness

- There is no state related to a particular client maintained on the server
  - In particular, **no session information** for a particular client IP maintained at server side
- Each request from the client must contain all required information to process the request
  - Session state (authentication token, identifier of working document, etc.) maintained by client and sent with each query
- Major advantage: scalability
  - Multiple instances, each able to process *any* client request
  - Add/remove server instances to follow demand
  - Fits well the serverless model, where each request is processed by a function without maintaining context between executions
- Example: In Heroku, *any* web dyno can process incoming request

Interest of using REST APIs

- Scalability and elasticity
  - Separation of concerns and statelessness allow using arbitrary number of server processes
  - Well adapted to the use of *caching* for GET requests
- Simplicity and portability
  - No assumption on server implementation or state
  - Effect of HTTP requests directly understandable from architecture point of view
- Using HTTP allows leveraging tools created for it: proxies, TLS/certificates (HTTPS), …
  - Firewall-friendly: ports 80/443 generally open

REST maturity model

- Richardson's model of maturity levels
  - Level 0: client invoke service by making HTTP POST requests to a single URL, with parameters in the body
  - Level 1: uses the notion of resource, but not HTTP verbs
    - Requests are like methods for an object
  - Level 2: use HTTP verbs to perform actions
    - Enables caching for GET requests
  - Level 3: *HATEOAS* (*Hypertext As The Engine Of Application State*)
    - The resource representation returned by a GET contains URL links for all actions allowed on that resource
- No all interfaces compliant with all REST guidelines
  - Still often called REST interfaces, and RESTful for levels 2 and 3

Example of RESTful APIs (I)

- IaaS management platform — OpenStack REST API
  - Programmatically launch VM instances, create images, etc.

GET	/v2.1/{tenant_id}/servers	**List servers** Lists IDs, names, and links for all servers.	detail
POST	/v2.1/{tenant_id}/servers	**Create server** Creates a server.	detail
GET	/v2.1/{tenant_id}/servers/detail	**List details for servers** Lists all servers with details.	detail
GET	/v2.1/{tenant_id}/servers/{server_id}	**Show server details** Shows details for a server.	detail
PUT	/v2.1/{tenant_id}/servers/{server_id}	**Update server** Updates the editable attributes of a server.	detail
DELETE	/v2.1/{tenant_id}/servers/{server_id}	**Delete server** Deletes a server.	detail

Example of use of OpenStack API

- Authenticating into Open Stack

```
curl -d '{"auth":{"passwordCredentials":{"username": "admin",
"password": "secret"},"tenantName": "customer-A"}}'
-H "Content-Type: application/json" http://192.168.100.100:5000/v2.1/tokens
```

  - Returns a large JSON representation
  - Includes a "Token ID" allowing to prove authentication
    - Remember there is no state about the session stored on server!

- List tenants (users your credentials allow your to see)

```
curl -H "X-Auth-Token: <Token ID>" http://192.168.100.70:5000/v2.1/tenants
```

  - Returns list of tenants as JSON representation of list of URIs

- List VMs for one particular user/tenant

```
curl -H "X-Auth-Token: <Token ID>" http://192.168.100.70:8774/v2.1/
<Tenant ID>/servers
```

- Give details about a particular VM

```
curl -H "X-Auth-Token: <Token ID>" http://192.168.100.70:8774/v2.1/
<Tenant ID>/servers/<Server ID>
```

Micro-services

Motivating example

- Building a taxi-hailing application
  - Web interface
  - Mobile interfaces for drivers and for clients
- How should you structure the backend of this application?
  - Simpler is better: start a new project with some existing Web/MVC framework (Rails, Spring, Django, …)
  - Write a core application with a set of modules implementing all the business logic
  - Adapters provide APIs, database access, and link to external services (SMS, payments, emails, …)

Our application

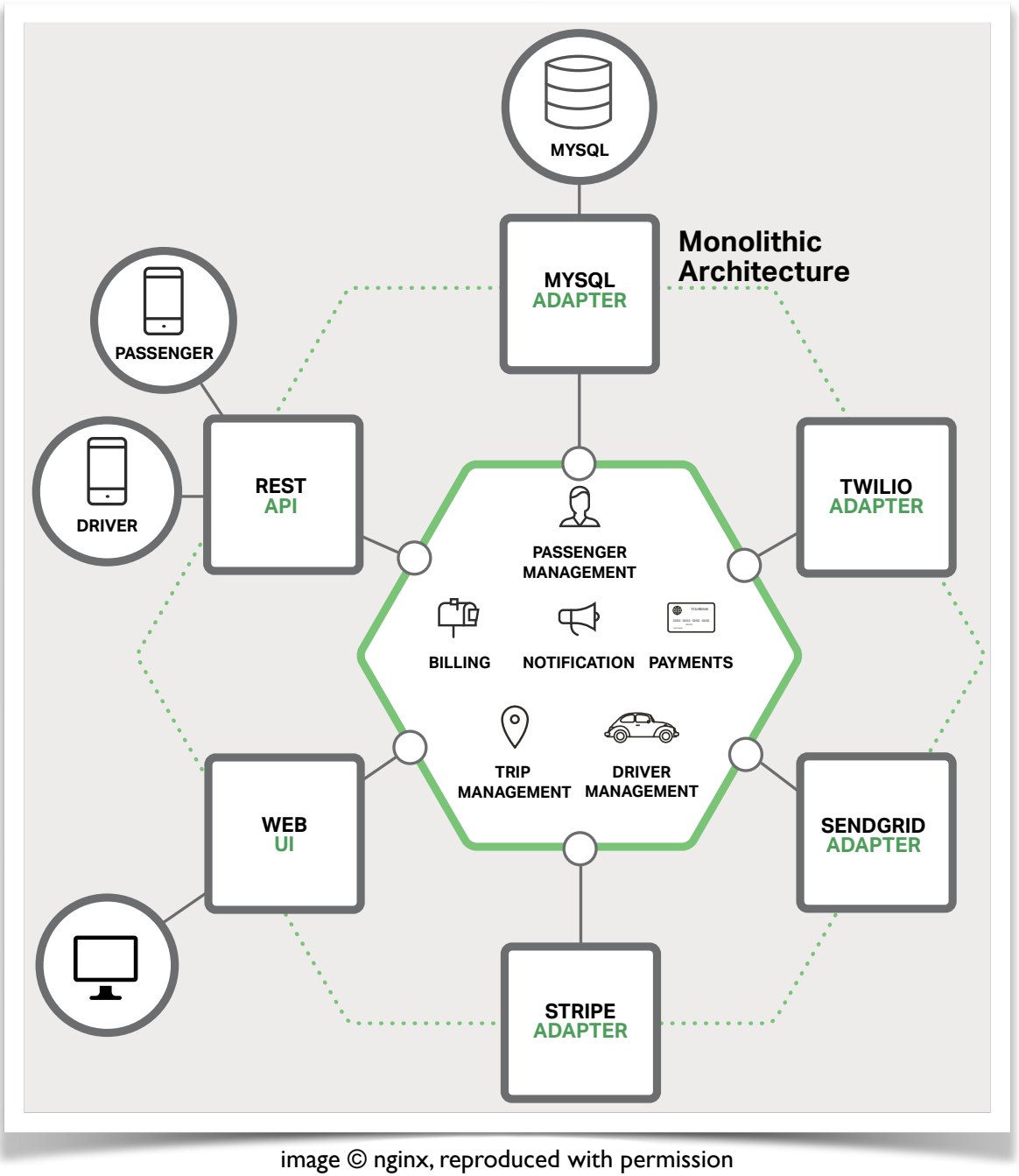


image © nginx, reproduced with permission

Advantages of a monolith (1)

- Application packaged as a single entity
  - Uses modules but all written in the same language (e.g. WAR files for Java application servers such as Tomcat)
  - The *layered* monolith
- Simple to deploy
  - Support more users by deploying more complete instances of the monolith behind a load balancer
- Tooling support
  - IDEs (Eclipse, IntelliJ) made for full application development

Monolithic Hell (1): dev perspective

- As our startup grows and attracts new clients
  - Code base grows as well
  - Eventually leads to a *monstrous monolith*



- Application now too complex and large to understand for a single developer
  - Adding new features and fixing bugs get increasingly complex and time consuming
- Very difficult to adopt new frameworks and languages
  - Would need to rewrite the entire application
  - Choice of initial language and framework is critical

Monolithic Hell (2): ops perspective

- Complexifies the use of an Agile development process and the quick release of new features
  - Requires redeploying the whole application
    - May lead to downtime, or to difficulties managing multiple versions deployed simultaneously
  - Large applications take time to test, deploy and bootstrap
- Deployment with conflicting resource requirements
  - Some modules may prefer compute-intensive instances, some others may need memory- or IO-intensive ones
  - Not possible to use different instances types, as modules deployed together: must select compromise or pay for more than we need

Microservices

- An architectural pattern
  - Split application in set of *small* and *interconnected* services
- Each microservice implements a single, distinct feature or functionality
  - With its own, private database
  - Offers an API for other services to consume
  - Developed and tested by different (small) teams
    - New frameworks and languages can co-exist for old and new services
  - Different services separately deployed incrementally
- A classical instantiation = containers + REST APIs

Our application as microservices

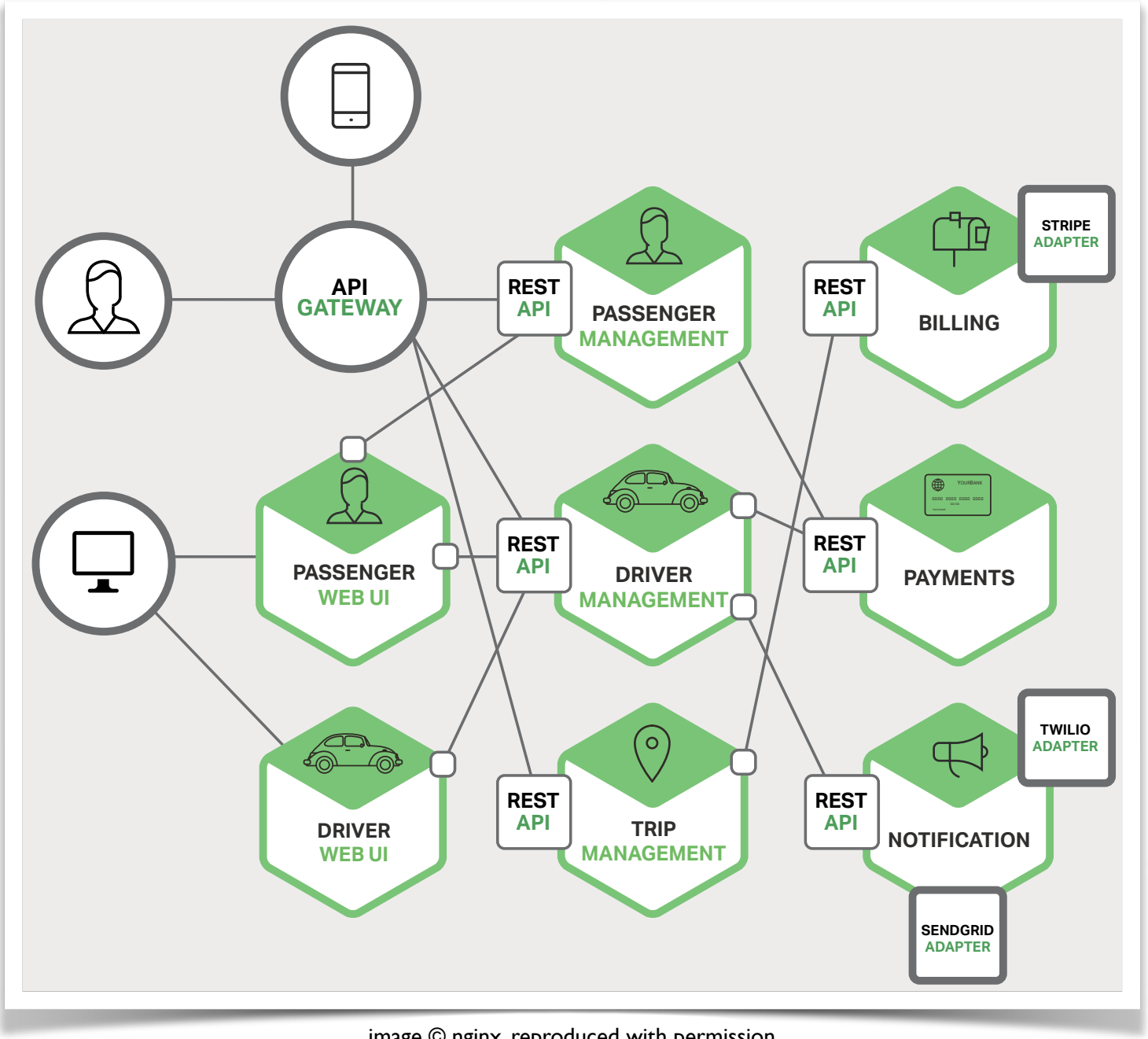


image © nginx, reproduced with permission

Interface

- Each functional area now implemented as a microservice
  - Even interfaces can be split for each type of client (passenger or driver)

- Backend services consume APIs provided by other services
  - e.g. Driver Management puts new message to Notification to send a SMS to a driver about newly available trip
  - Can also use asynchronous event-based communication

The scalability cube

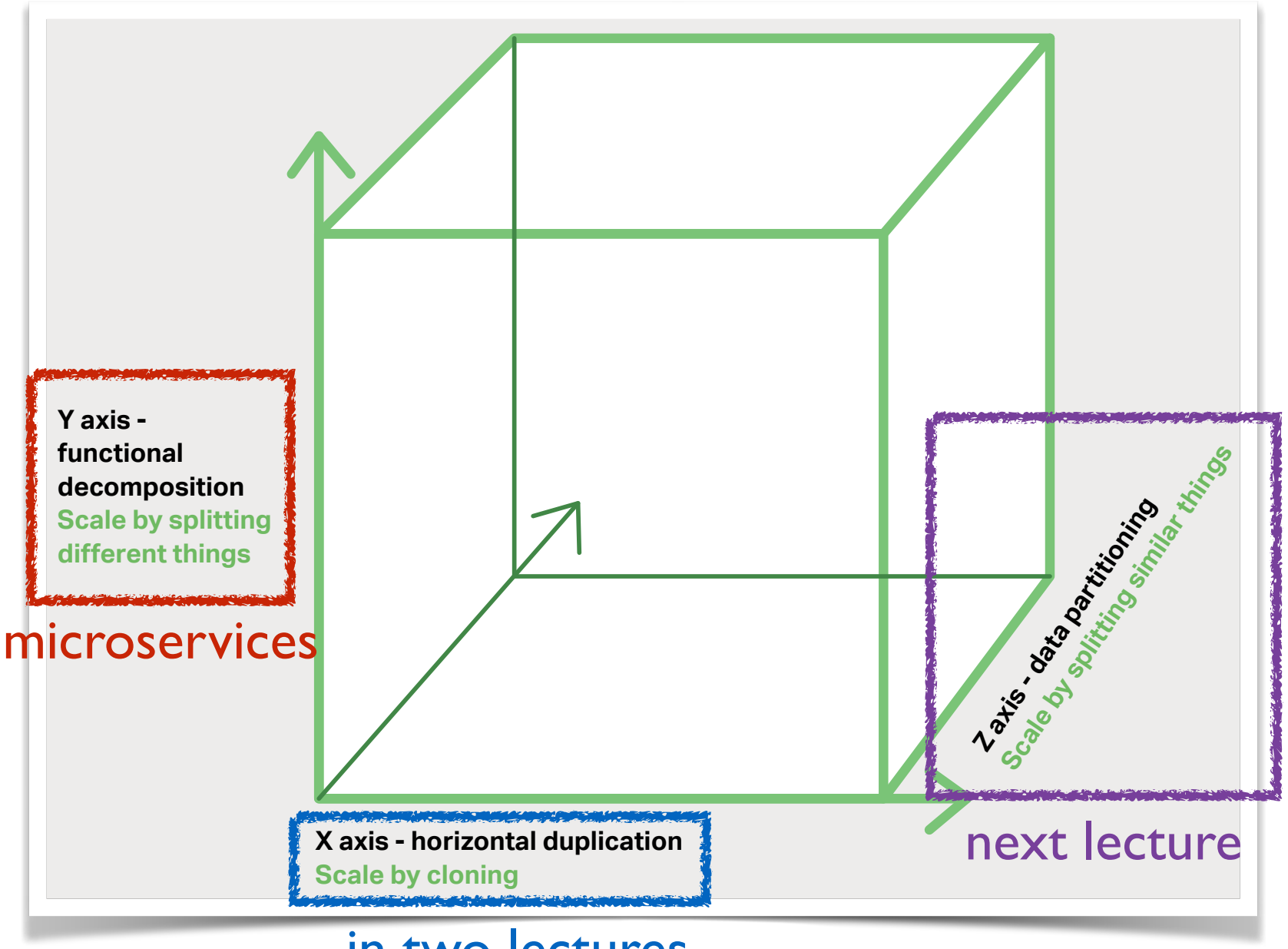


image © nginx, reproduced with permission

Towards elastic scaling

- REST *interfaces* are stateless
  - Can use a number of service instances behind a load balancer
  - If the service implementation itself is stateful, need state sharing between instances (using a common database)

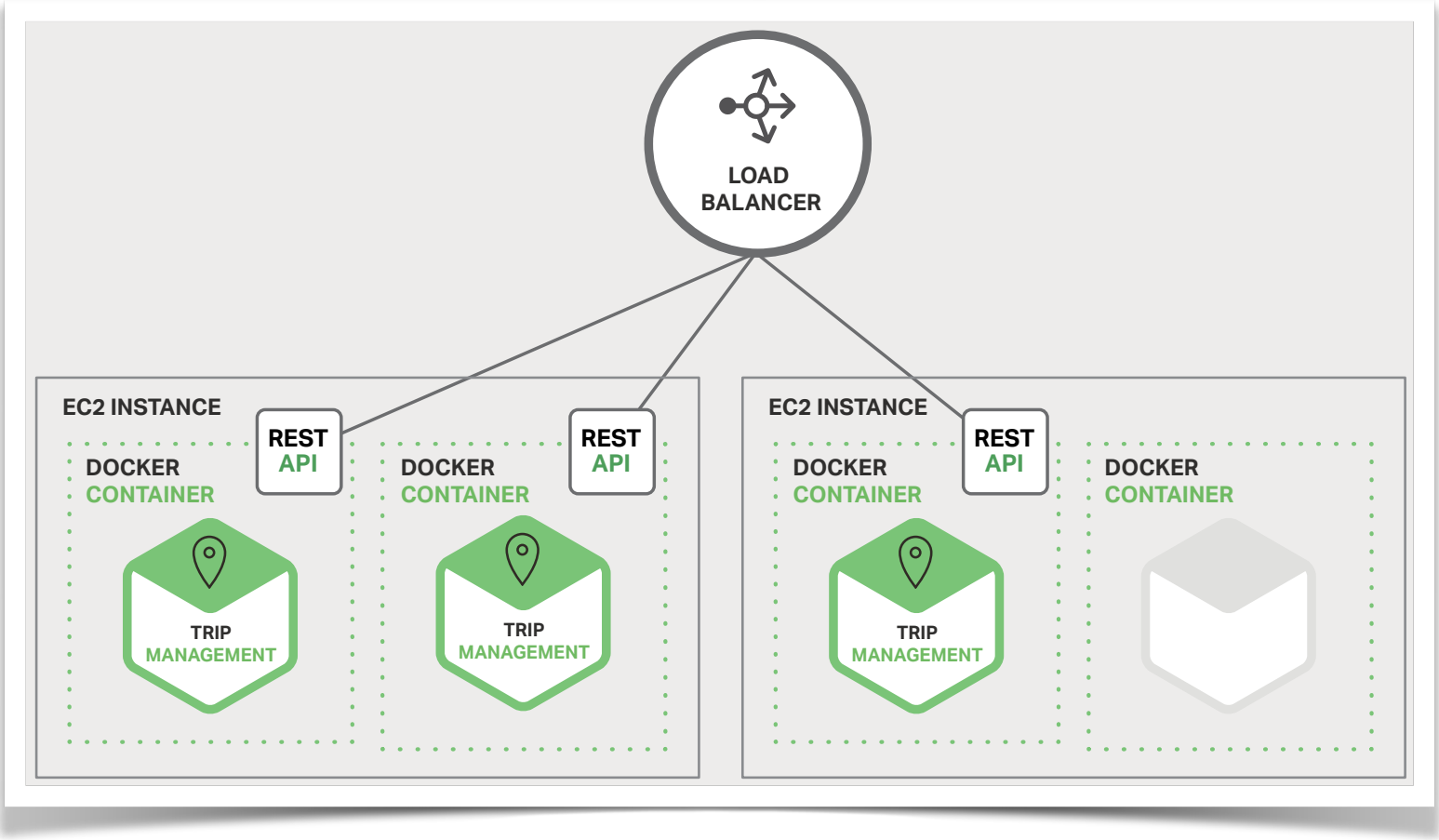


image © nginx, reproduced with permission

Microservices and databases

- Each microservice uses its own database
  - There is no longer a global, enterprise-wide database or data model
  - ☹ Some duplication of data
  - 😀 Each microservice can choose favored database (polyglot architecture)
    - e.g. Driver Management must find drivers close to passengers: spatial databases have excellent support for such queries

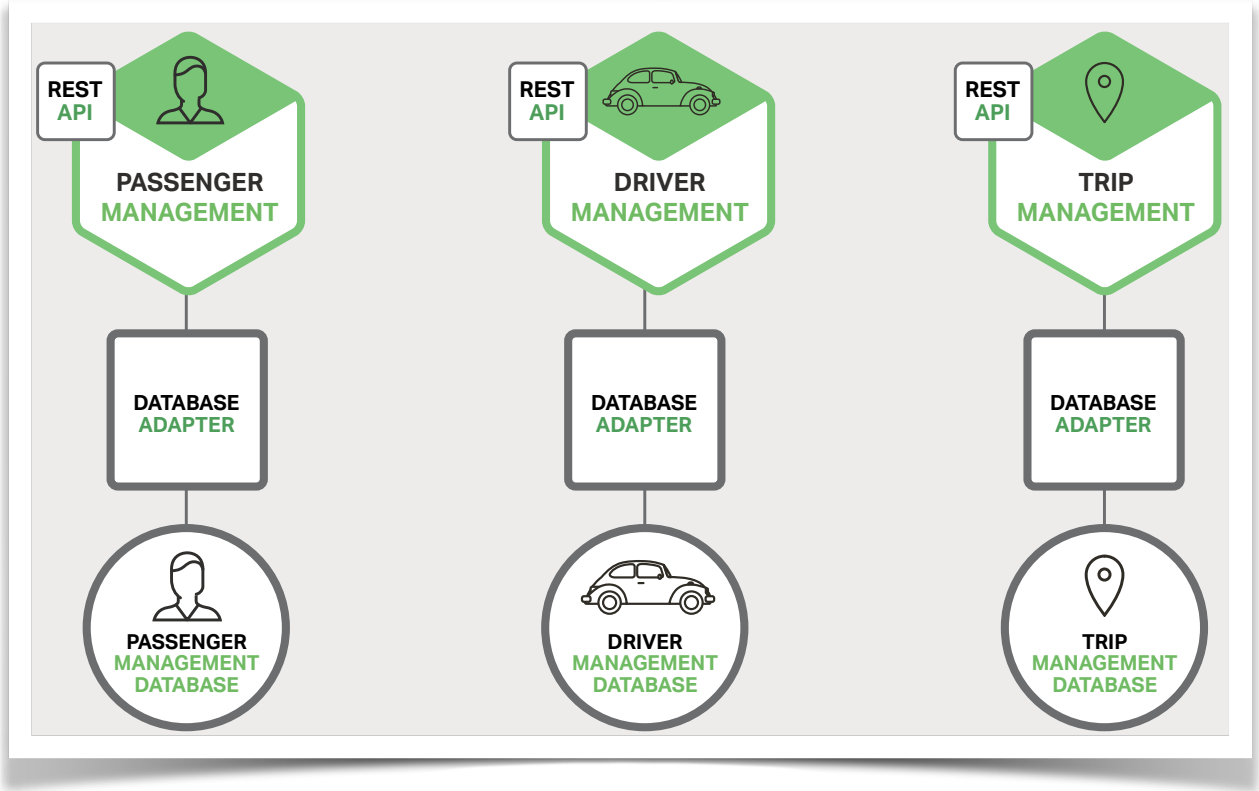
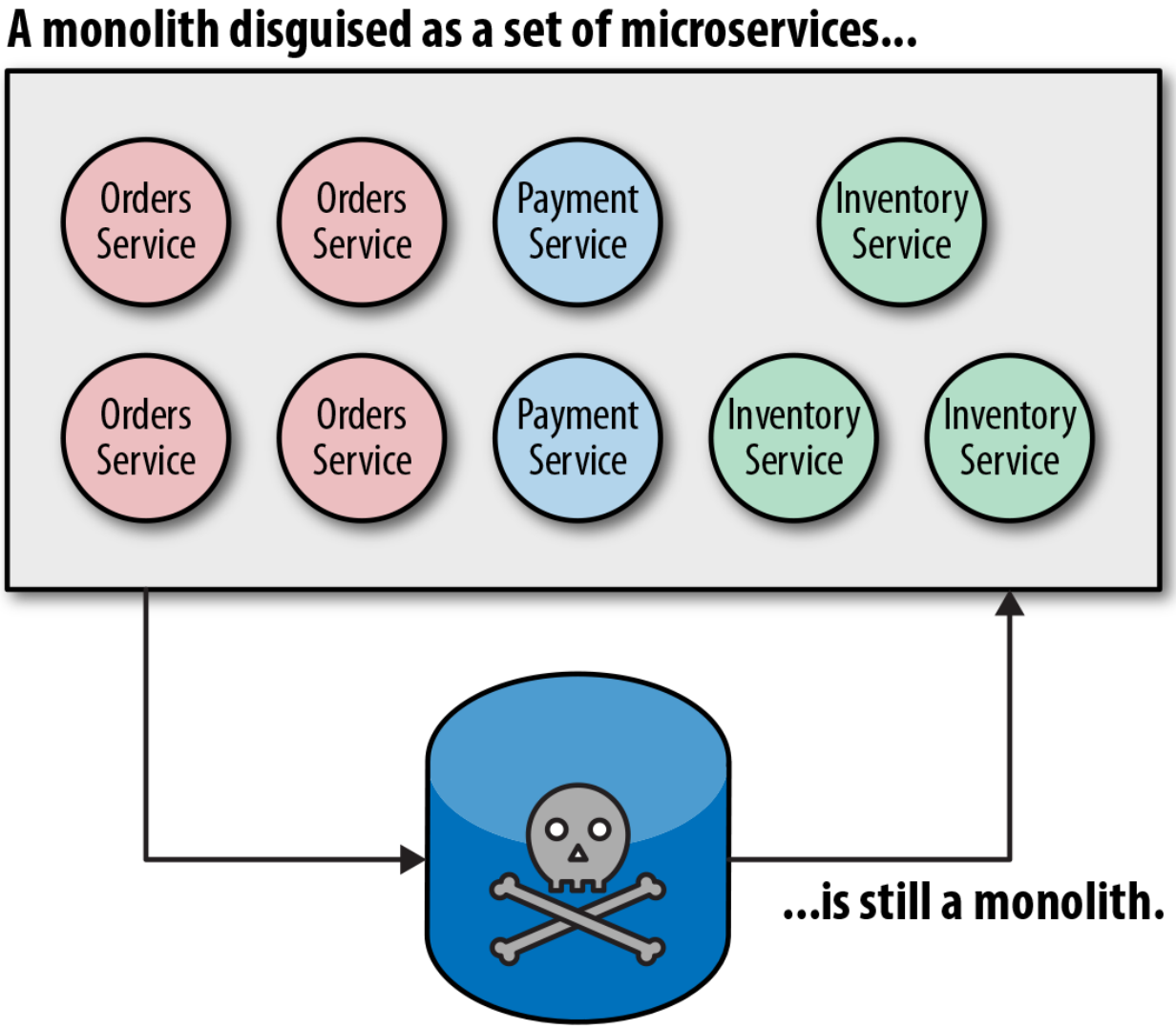


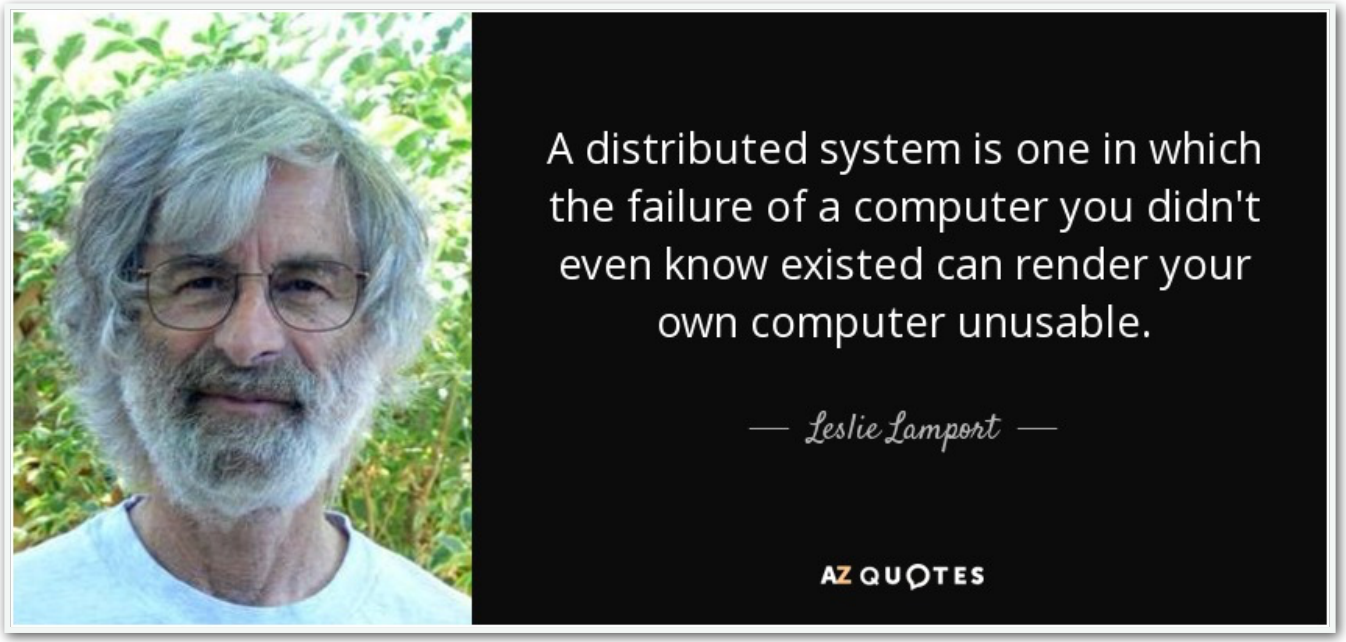
image © nginx, reproduced with permission

Beware ...


A monolith disguised as a set of microservices...
Orders Service
Orders Service
Payment Service
Inventory Service
Orders Service
Orders Service
Payment Service
Inventory Service
Inventory Service
...is still a monolith.

Complexities of microservices

- Your application is now a distributed system



  - No shared state means we need interprocess communication (RPC) or event-based (asynchronous) communication
  - Need to build around faults

Complexities of microservices

- Partitioned database architecture
  - No longer possible to use *transactions* that modify multiple database entries across multiple tables *atomically* (i.e. in an *all-or-nothing* fashion)
  - Need to implement changes across multiple databases explicitly using interactions between microservices
    - What about faults occurring in the process and their impact on the consistency of multiple services' states?
- Testing and incremental deployment more complex
  - Must handle these as code and not manually (DevOps)
  - Runtime support for discovery and configuration
  - PaaS platforms (e.g., CloudFoundry) built-in support for micro-service deployment, or use Kubernetes

Communication

- How to connect a mobile/native (front-end) app client to our microservices?
- How should our backend microservices interact?

Communication: direct from client

- Client could call the different microservices
  - Know and call a set of public endpoints:
    - `https://serviceName.api.company.name`
- Drawbacks
  - Rendering an Amazon page = 100s of microservices
  - ☹ Many requests over a wide-are network (4G/5G)
  - ☹ Client code get very complex and dependent on the internal architecture
  - ☹ Any modification (fusion, fission, rename) to the set of services requires updating the client application!

Communication: direct from client

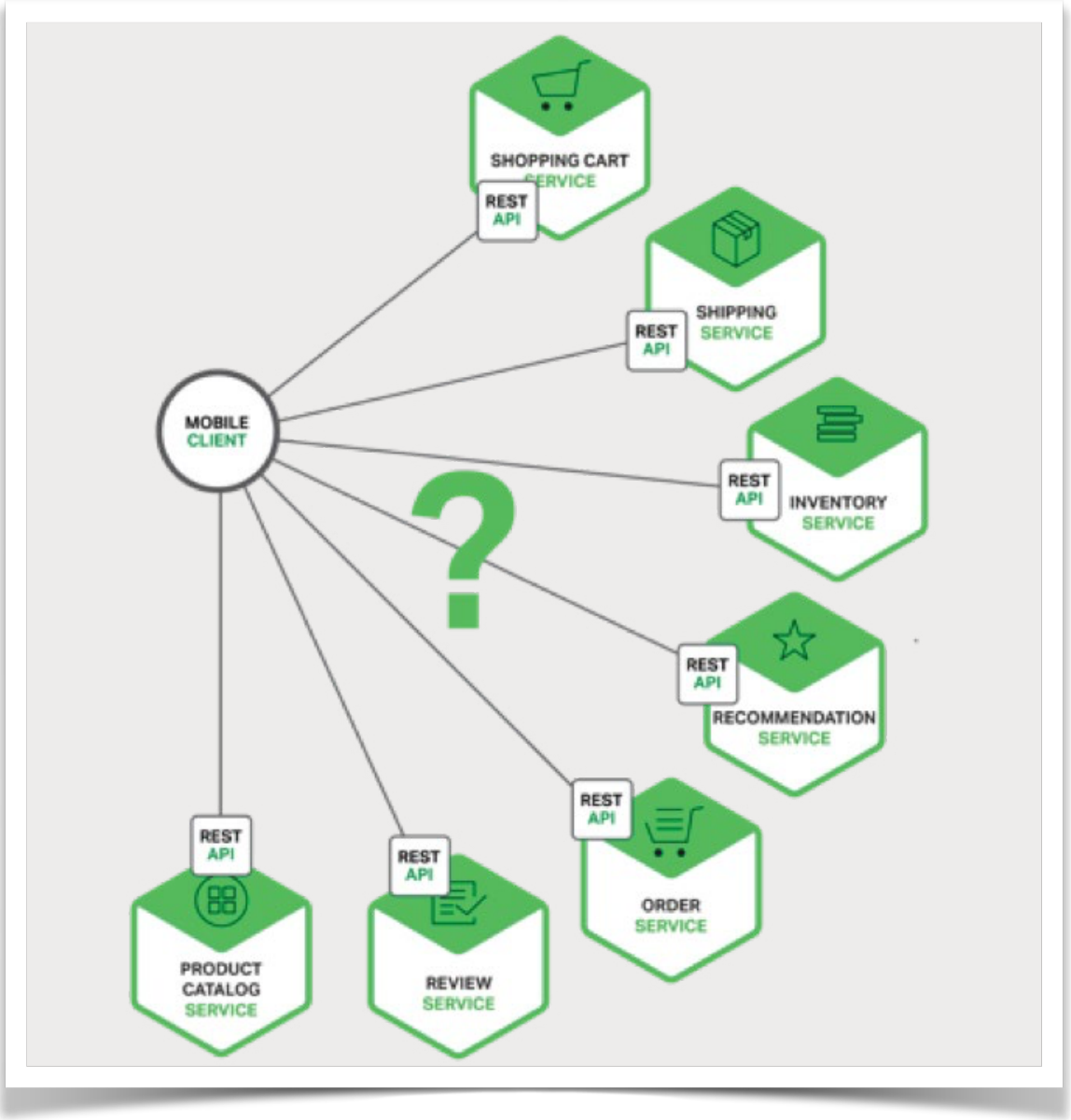


image © nginx, reproduced with permission

Using an API Gateway

- Specific microservice acting as single entry point
  - Encapsulates knowledge of internal system architecture
  - Provides an API for each type of client
    - Example: Netflix API gateway
    - Orchestrates microservices differently depending on device
    - 6 or 7 microservices called for each client request on average, and sometimes many more
  - May also ensure common functions such as
    - Authentication
    - Monitoring
    - Load balancing
    - DOS protection (IP-based throttling)

Using an API Gateway

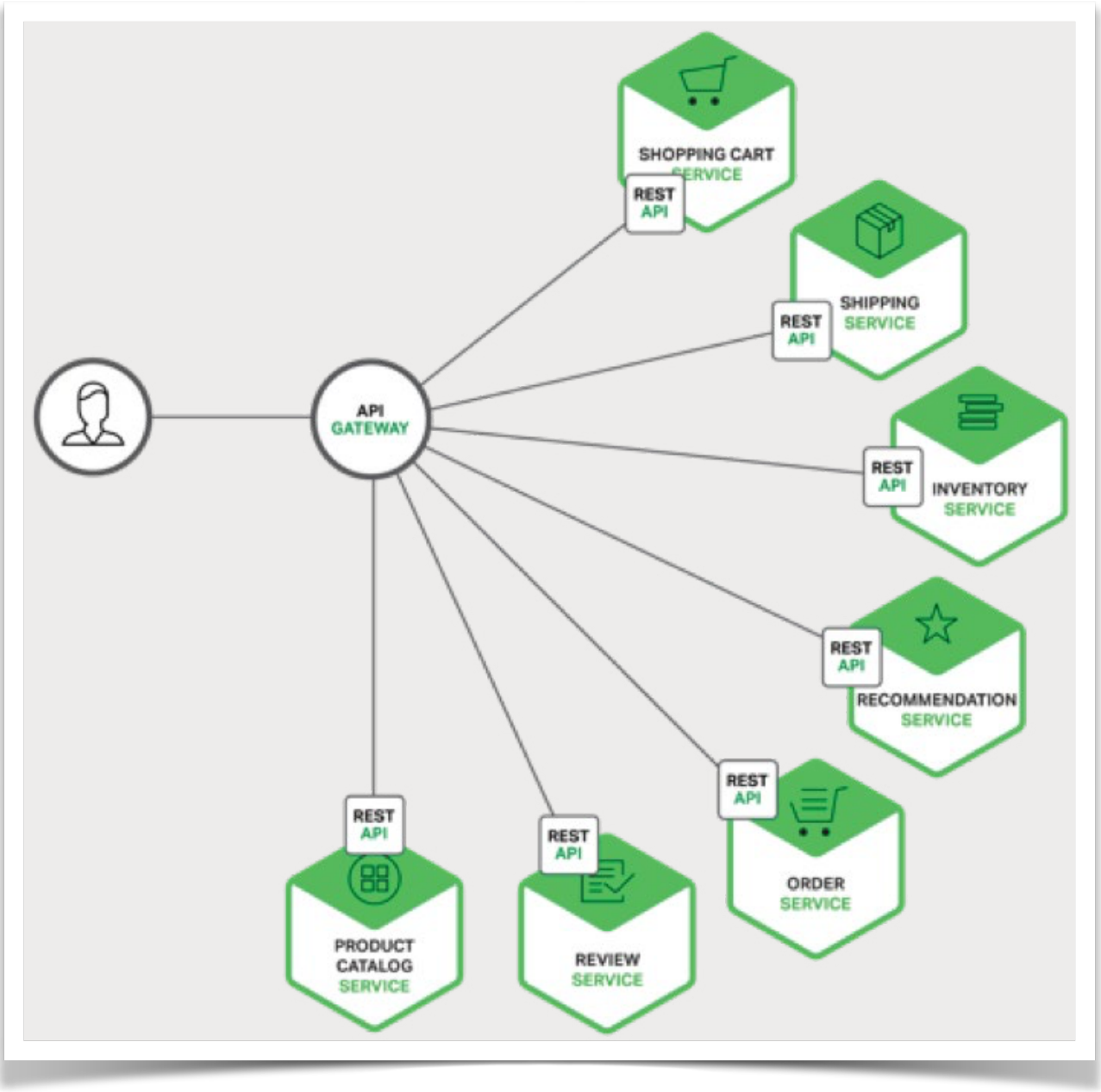


image © nginx, reproduced with permission

Communication between microservices

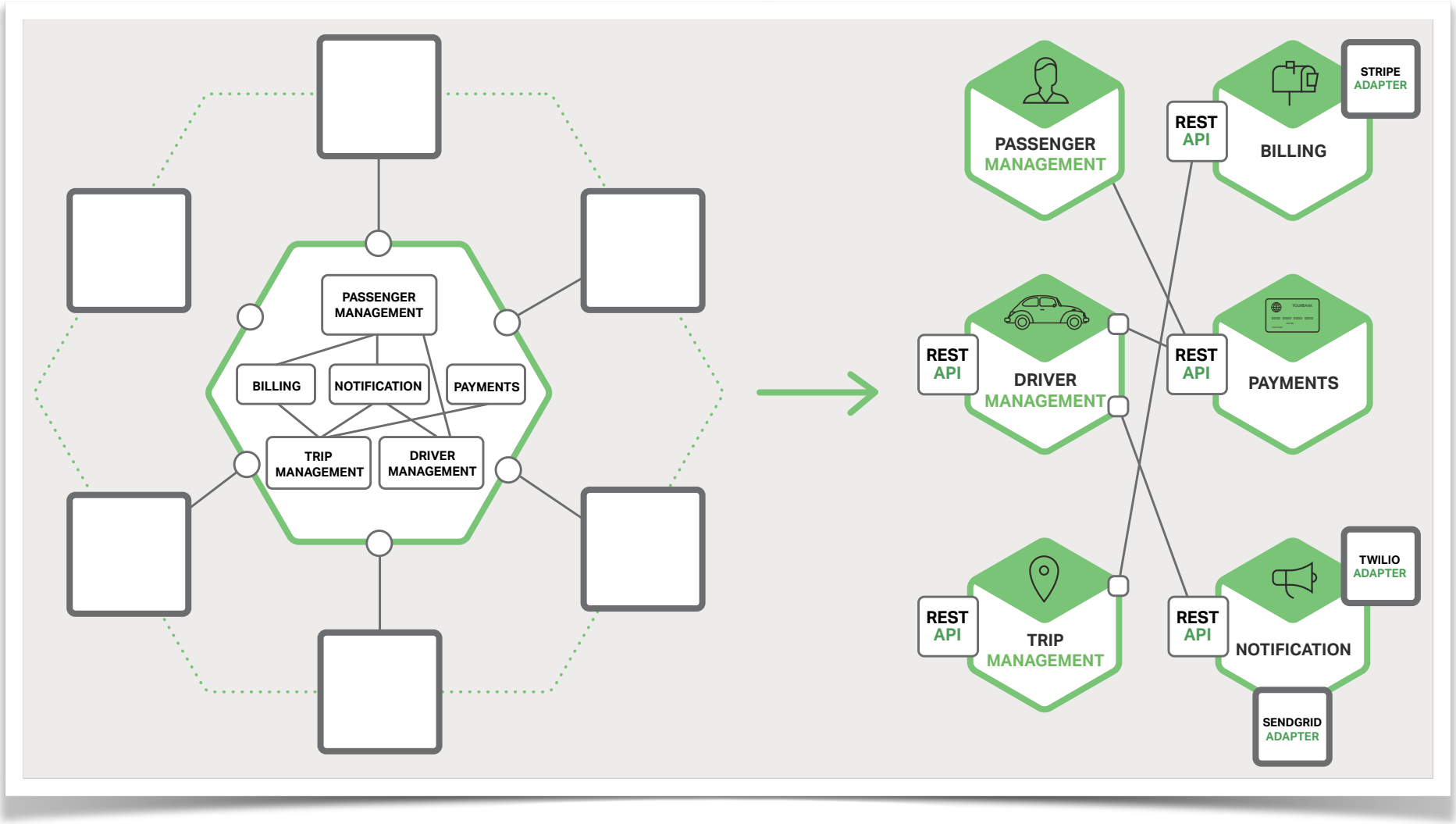


image © nginx, reproduced with permission

Interaction styles

- One-to-one
  - Each request processed by exactly one service instance
- One-to-many
  - Each request is processed by instances of multiple services
- Synchronous
  - Blocking call, need fast response
- Asynchronous
  - Client does not block waiting for response
    - May poll for response later, if necessary (2nd line)

	ONE-TO-ONE	ONE-TO-MANY
SYNCHRONOUS	Request/response	—
ASYNCHRONOUS	Notification	Publish/subscribe
	Request/async response	Publish/async responses

Mixing interaction styles

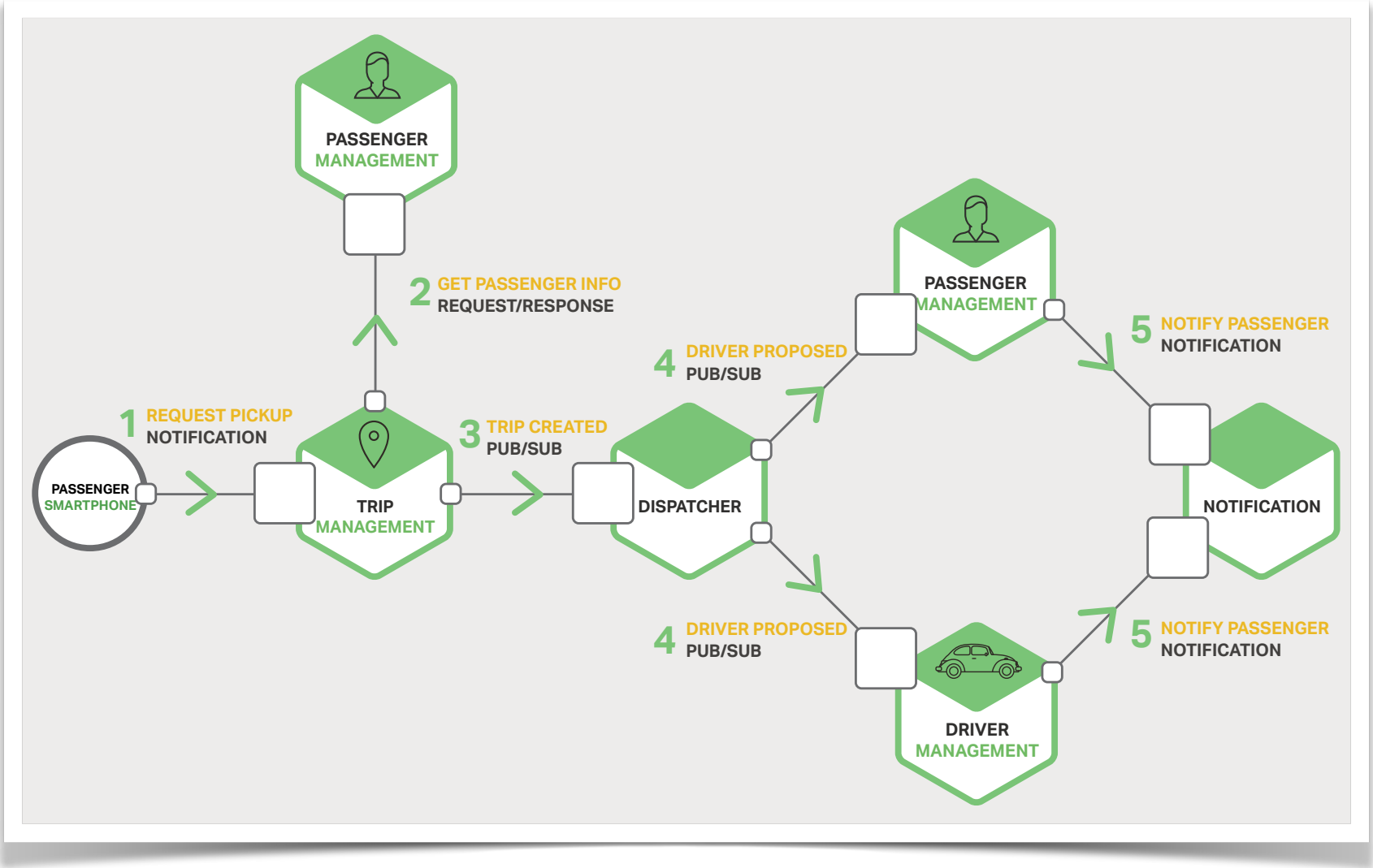
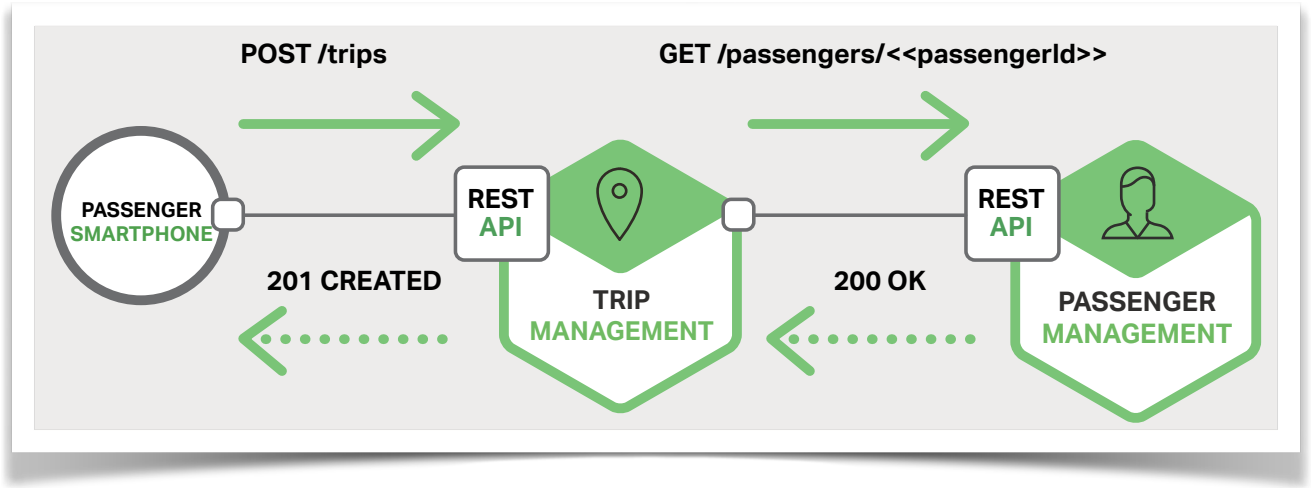


image © nginx, reproduced with permission

Synchronous request/response



- REST over HTTP: common choice but inefficient for a data-center setting (large plaintext messages, parsing overhead, etc.)
- Apache Thrift: cross-language RPC library
  - C-style Interface Definition Language
    - Think of Java Interfaces
  - Can use TCP directly instead of HTTP: faster
  - OK for internal communication but **not** to/from client
    - Gateway service typically handles the translation

image © nginx, reproduced with permission

Asynchronous, event-based communication

- Client makes request to service using an event
  - Metadata + payload
- Response (if any) received later as another event
  - Client does not block for reply
- Events are exchanged over a *channel*
  - Point-to-point: to exactly one event consumer
    - Both endpoints identified when creating the channel
  - Publish/subscribe: event consumed by multiple services
    - Consumers *subscribe* to a channel
    - Clients *publish* to the channel

Asynchronous, event-based communication

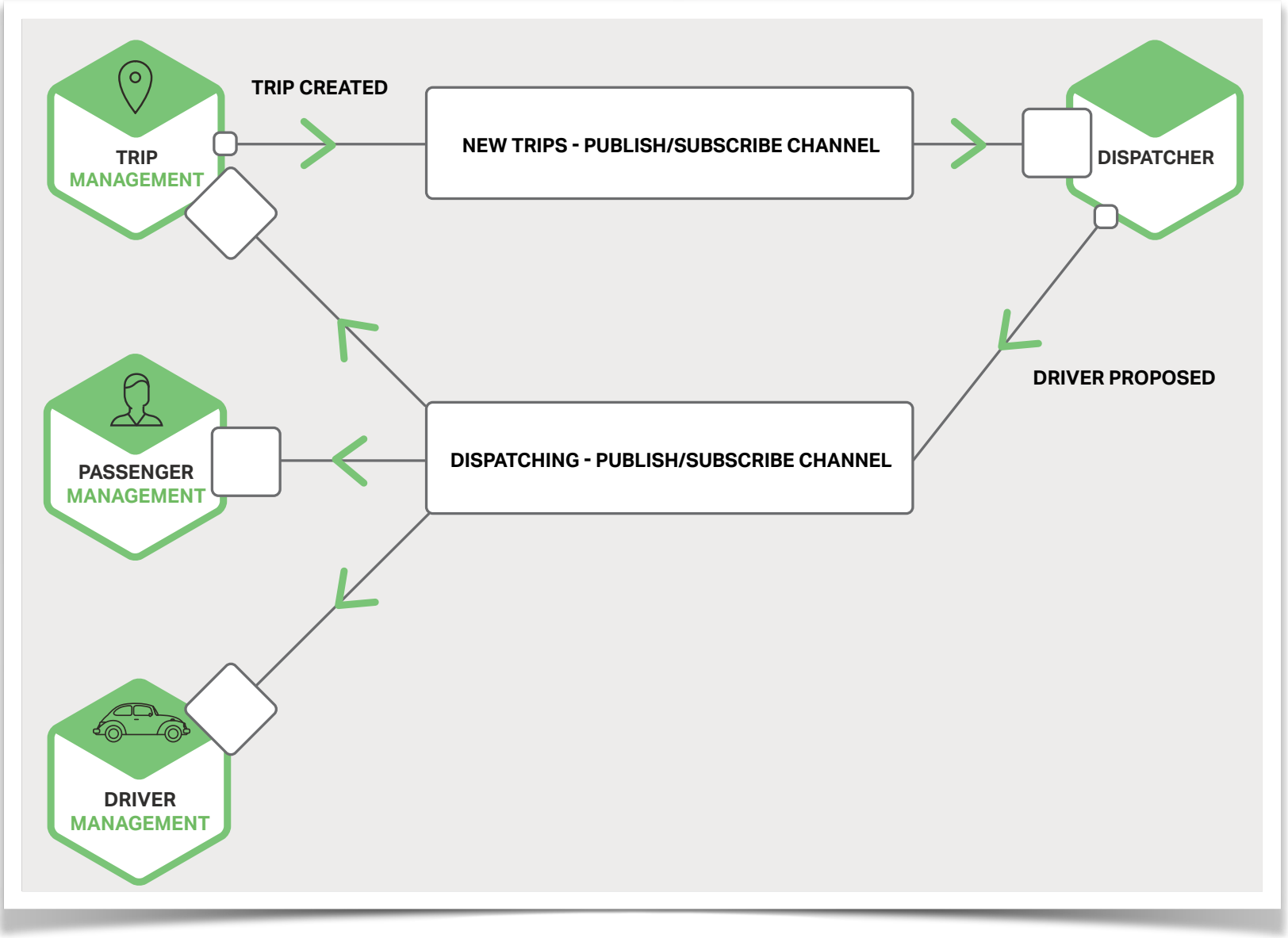


image © nginx, reproduced with permission

Asynchronous, event-based communication

- Many open-source messaging systems
  - RabbitMQ, Apache Kafka, ActiveMQ, etc.







- Service decoupling
  - 😀 Client does not need to know where the service instance is located: simply push to a channel
  - 😀 Message buffering: performance
  - ☹ More complex to implement request/response

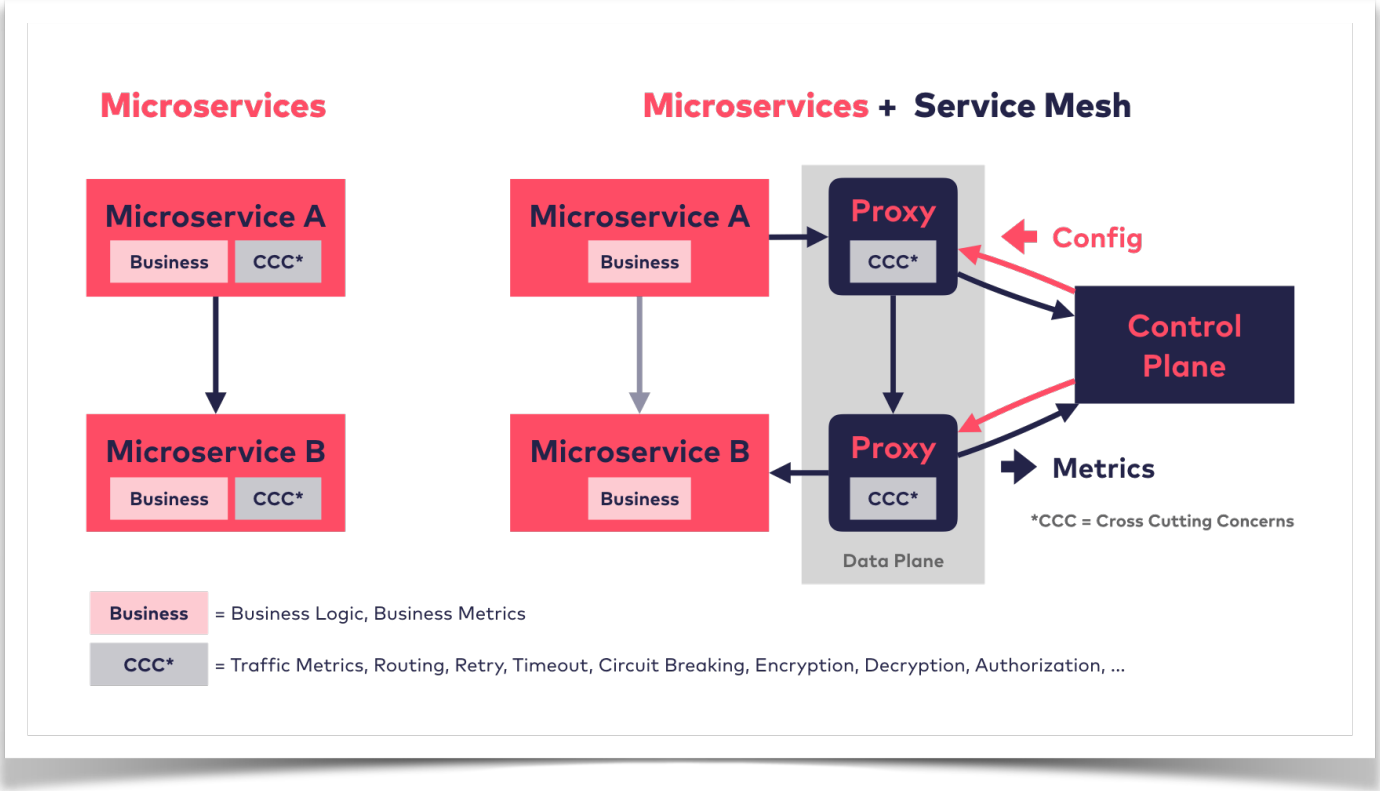
Payloads

- Interactions exchange data
  - Need a common representation
  - Conflicting goals:
    fast parsing, easy debugging, space-efficiency
- Message formats
  - Human-readable: JSON or XML
  - Binary: Apache Avro, Google Protocol Buffers
    - More efficient but make debugging more complex
    - May need the use of a shared (fixed) schema
- Binary format OK to use internally but again not suitable for communication between client app and backend





Service Meshs



- Help with cross-cutting concerns (CCC) such as collecting performance or faults metrics, ensuring the use of TLS, etc.
- Generally implemented using companion containers on each node or for each microservice instance, plus a centralized controller ("Control Plane")
- Examples: Istio for Kubernetes, Consul, Open Service Mesh (OSM) by Microsoft
- More details on https://servicemesh.es (link on Moodle)

image © INNOQ, reproduced with permission

Conclusion

- PaaS Clouds offer a development and application runtime environment
  - "Serverless" model allows running functions upon request, useful for low-traffic applications
- Service-oriented architectures
  - Well suited to continuous integration and DevOps
  - Typically deployed as containers hosting microservices, either in a managed or unmanaged cloud (IaaS or PaaS), or as serverless functions
  - Will form the backend of the project

Additional resources

- Watch video by Netflix's Josh Evans
  - Link on Moodle
  - **Mandatory**

- Optional: e-Book available on Moodle
  - On microservices from NGINX
  - Chapters 1-3 cover the second half of lecture

- Optional: INNOQ webpage and e-Book about service mesh principles and systems

References

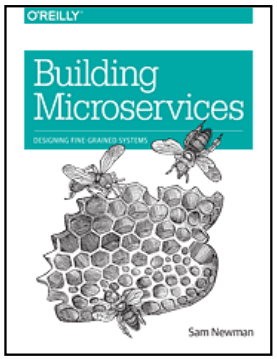
Cesare Pautasso, Olaf Zimmermann, Frank Leymann:
Restful web services vs. "big"' web services: making the right architectural decision. WWW 2008: 805-814

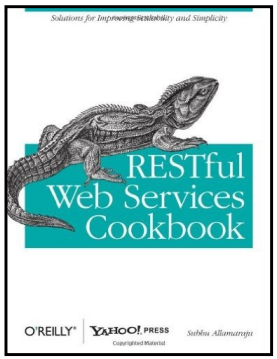
REST API Design Rulebook - Designing Consistent RESTful Web Service Interfaces By Mark Masse,
Publisher: O'Reilly Media,
Final Release Date: October 2011 Pages: 116

RESTful Web Services Cookbook
By Subbu Allamaraju
Publisher: O'Reilly Media

Building Microservices
By Sam Newman
Publisher: O'Reilly Media









Exploring CQRS and Event Sourcing, A journey into high scalability, availability, and maintainability with Windows Azure (Dominic Betts, Julián Domínguez, Grigori Melnik, Fernando Simonazzi and Mani Subramanian), Microsoft, 2017

Microservices, from Design to Deployment (Chris Richardson and Floyd Smith), NGINX, 2018